```
310 |     it('should handle template rendering errors gracefully', async () => {
311 |       const invalidTemplate: ContextTemplate = {
312 |         id: 'invalid-template',
313 |         name: 'Invalid Template',
314 |         description: 'Template with invalid syntax',
315 |         techStack: ['test'],
316 |         template: 'Invalid template: {{unclosedVariable',
317 |         variables: [],
318 |         outputFormat: 'text'
319 |       };
320 |
321 |       contextEnhancer.registerTemplate(invalidTemplate);
322 |
323 |       const options: ContextOptions = {
324 |         templateId: 'invalid-template'
325 |       };
326 |
327 |       // Should not throw, but handle gracefully
328 |       const context = await contextEnhancer.generateContext(mockContextData,
options);
329 |       expect(context).toBeDefined();
330 |     });
331 |   });
332 |
333 |   describe('statistics', () => {
334 |     it('should provide accurate statistics', async () => {
335 |       await contextEnhancer.generateContext(mockContextData);
336 |
337 |       // Wait a bit to ensure different timestamp
338 |       await new Promise(resolve => setTimeout(resolve, 10));
339 |
340 |       await contextEnhancer.generateContext({
341 |         ...mockContextData,
342 |         projectInfo: {
343 |           ...mockContextData.projectInfo,
344 |           name: 'stats-test-project'
345 |         }
346 |       });
347 |
348 |       const stats = contextEnhancer.getStats();
349 |       expect(stats.templatesLoaded).toBeGreaterThan(0);
350 |       expect(stats.cachedContexts).toBe(2);
351 |       expect(stats.templatesPath).toBe(tempDir);
352 |     });
353 |   });
354 | });
355 |
```

```
--------------------------------------------------------------------------------
/tests/unit/cursor-adapter.test.ts:
--------------------------------------------------------------------------------
  1 | /**
  2 |  * Unit tests for Cursor Adapter
  3 |  */
  4 |
  5 | import { CursorAdapter, CursorAdapterConfig, CursorValidationResult,
CursorContextData } from '../../src/adapters/cursor';
  6 | import * as vscode from 'vscode';
  7 | import { EventEmitter } from 'events';
  8 |
  9 | // Mock the core modules
 10 | jest.mock('../../src/core/validation', () => ({
 11 |   ValidationEngine: jest.fn().mockImplementation(() => ({
 12 |     initialize: jest.fn().mockResolvedValue(undefined),
 13 |     validateInterface: jest.fn().mockResolvedValue({
 14 |       isValid: true,
 15 |       errors: [],
 16 |       warnings: [],
 17 |       suggestions: []
 18 |     })
 19 |   }))
 20 | }));
 21 |
 22 | jest.mock('../../src/core/context', () => ({
 23 |   ContextEnhancer: jest.fn().mockImplementation(() => ({
 24 |     initialize: jest.fn().mockResolvedValue(undefined)
 25 |   }))
 26 | }));
 27 |
 28 | jest.mock('../../src/core/parser', () => ({
 29 |   SpecParser: jest.fn().mockImplementation(() => ({
 30 |     parseSpec: jest.fn().mockResolvedValue({
 31 |       success: true,
 32 |       spec: { openapi: '3.0.0' },
 33 |       errors: []
 34 |     })
 35 |   }))
 36 | }));
 37 |
 38 | jest.mock('../../src/utils/logger', () => ({
 39 |   Logger: jest.fn().mockImplementation(() => ({
 40 |     info: jest.fn(),
 41 |     error: jest.fn(),
```

```
42 |     warn: jest.fn(),
43 |     debug: jest.fn()
44 |   }))
45 | }));
46 |
47 | // Mock vscode module
48 | jest.mock('vscode', () => ({
49 |   workspace: {
50 |     onDidChangeTextDocument: jest.fn(() => ({ dispose: jest.fn() })),
51 |     onDidSaveTextDocument: jest.fn(() => ({ dispose: jest.fn() }))
52 |   },
53 |   window: {
54 |     onDidChangeActiveTextEditor: jest.fn(() => ({ dispose: jest.fn() })),
55 |     activeTextEditor: undefined,
56 |     showWarningMessage: jest.fn(),
57 |     showErrorMessage: jest.fn(),
58 |     showInformationMessage: jest.fn()
59 |   },
60 |   commands: {
61 |     registerCommand: jest.fn(() => ({ dispose: jest.fn() }))
62 |   },
63 |   Position: jest.fn(),
64 |   Selection: jest.fn(),
65 |   Uri: {
66 |     parse: jest.fn()
67 |   }
68 | }));
69 |
70 | describe('CursorAdapter', () => {
71 |   let adapter: CursorAdapter;
72 |   let mockConfig: CursorAdapterConfig;
73 |   let mockOutputChannel: any;
74 |
75 |   beforeEach(async () => {
76 |     // Reset all mocks
77 |     jest.clearAllMocks();
78 |
79 |     // Create mock output channel
80 |     mockOutputChannel = {
81 |       appendLine: jest.fn(),
82 |       show: jest.fn(),
83 |       dispose: jest.fn()
84 |     };
85 |
86 |     // Create test configuration
87 |     mockConfig = {
88 |       enableRealTimeValidation: true,
```

```
 89 |       contextInjectionMode: 'auto',
 90 |       validationTriggers: ['typescript', 'javascript', 'json', 'yaml'],
 91 |       outputChannel: mockOutputChannel,
 92 |       workspaceRoot: '/test/workspace'
 93 |     };
 94 |
 95 |     // Create adapter instance
 96 |     adapter = new CursorAdapter(mockConfig);
 97 |   });
 98 |
 99 |   afterEach(() => {
100 |     if (adapter) {
101 |       adapter.dispose();
102 |     }
103 |   });
104 |
105 |   describe('initialization', () => {
106 |     it('should initialize successfully', async () => {
107 |       const initPromise = adapter.initialize();
108 |
109 |       // Listen for initialized event
110 |       const initPromiseWithEvent = new Promise<void>((resolve) => {
111 |         adapter.once('initialized', resolve);
112 |       });
113 |
114 |       await initPromise;
115 |       await initPromiseWithEvent;
116 |
117 |       expect(adapter.getStatistics().isActive).toBe(true);
118 |     });
119 |
120 |     it('should set up event listeners when real-time validation is enabled',
async () => {
121 |       await adapter.initialize();
122 |
123 |       expect(vscode.workspace.onDidChangeTextDocument).toHaveBeenCalled();
124 |       expect(vscode.workspace.onDidSaveTextDocument).toHaveBeenCalled();
125 |       expect(vscode.window.onDidChangeActiveTextEditor).toHaveBeenCalled();
126 |     });
127 |
128 |     it('should register commands', async () => {
129 |       await adapter.initialize();
130 |
131 |       expect(vscode.commands.registerCommand).toHaveBeenCalledWith(
132 |         'cursor-adapter.validateDocument',
133 |         expect.any(Function)
134 |       );
```

```
135 |       expect(vscode.commands.registerCommand).toHaveBeenCalledWith(
136 |         'cursor-adapter.injectContext',
137 |         expect.any(Function)
138 |       );
139 |       expect(vscode.commands.registerCommand).toHaveBeenCalledWith(
140 |         'cursor-adapter.clearCache',
141 |         expect.any(Function)
142 |       );
143 |     });
144 |   });
145 |
146 |   describe('document validation', () => {
147 |     beforeEach(async () => {
148 |       await adapter.initialize();
149 |     });
150 |
151 |     it('should validate API specification document', async () => {
152 |       const contextData: CursorContextData = {
153 |         documentUri: 'file:///test/api.yaml',
154 |         content: `
155 | openapi: 3.0.0
156 | info:
157 |   title: Test API
158 |   version: 1.0.0
159 | paths:
160 |   /users:
161 |     get:
162 |       responses:
163 |         '200':
164 |           description: Success
165 |         `,
166 |         language: 'yaml'
167 |       };
168 |
169 |       const result = await adapter.validateDocument(contextData);
170 |
171 |       expect(result).toBeDefined();
172 |       expect(result.timestamp).toBeGreaterThan(0);
173 |       expect(typeof result.isValid).toBe('boolean');
174 |       expect(Array.isArray(result.errors)).toBe(true);
175 |       expect(Array.isArray(result.warnings)).toBe(true);
176 |       expect(Array.isArray(result.suggestions)).toBe(true);
177 |     });
178 |
179 |     it('should handle non-API documents', async () => {
180 |       const contextData: CursorContextData = {
181 |         documentUri: 'file:///test/regular.ts',
```

```
182 |         content: `
183 | const hello = 'world';
184 | console.log(hello);
185 |         `,
186 |         language: 'typescript'
187 |       };
188 |
189 |       const result = await adapter.validateDocument(contextData);
190 |
191 |       expect(result).toBeDefined();
192 |         expect(result.isValid).toBe(true); // Non-API documents should pass
validation
193 |     });
194 |
195 |     it('should cache validation results', async () => {
196 |       const contextData: CursorContextData = {
197 |         documentUri: 'file:///test/api.json',
198 |          content: '{"openapi": "3.0.0", "info": {"title": "Test", "version":
"1.0.0"}}',
199 |         language: 'json'
200 |       };
201 |
202 |       // First validation
203 |       const result1 = await adapter.validateDocument(contextData);
204 |       const stats1 = adapter.getStatistics();
205 |
206 |       // Second validation with same content
207 |       const result2 = await adapter.validateDocument(contextData);
208 |       const stats2 = adapter.getStatistics();
209 |
210 |       expect(result1.timestamp).toBe(result2.timestamp); // Should be cached
211 |       expect(stats2.cacheSize).toBeGreaterThan(0);
212 |     });
213 |
214 |     it('should handle validation errors gracefully', async () => {
215 |       // Test with content that will trigger an exception
216 |       const contextData: CursorContextData = {
217 |         documentUri: 'file:///test/invalid.yaml',
218 |         content: 'openapi: 3.0.0\ninvalid: yaml: content: [unclosed',
219 |         language: 'yaml'
220 |       };
221 |
222 |       // Mock the parser to throw an error
223 |       const originalParseSpec = adapter['parser'].parseSpec;
224  |             adapter['parser'].parseSpec  =  jest.fn().mockRejectedValue(new
Error('Parse error'));
225 |
```

```
226 |       const result = await adapter.validateDocument(contextData);
227 |
228 |       expect(result.isValid).toBe(false);
229 |       expect(result.errors.length).toBeGreaterThan(0);
230 |       expect(result.errors[0]).toContain('Validation error');
231 |
232 |       // Restore original method
233 |       adapter['parser'].parseSpec = originalParseSpec;
234 |     });
235 |   });
236 |
237 |   describe('context injection', () => {
238 |     beforeEach(async () => {
239 |       await adapter.initialize();
240 |     });
241 |
242 |     it('should handle missing active editor', async () => {
243 |       // Mock no active editor
244 |       (vscode.window as any).activeTextEditor = undefined;
245 |
246 |       await adapter.injectContextToDocument();
247 |
248 |           expect(vscode.window.showWarningMessage).toHaveBeenCalledWith('No
active editor found');
249 |     });
250 |
251 |     it('should emit contextInjected event when context is injected', async ()
=> {
252 |       // Mock active editor
253 |       const mockEditor = {
254 |         document: {
255 |           uri: { toString: () => 'file:///test/api.ts' },
256 |           getText: () => 'const api = {};',
257 |           languageId: 'typescript'
258 |         },
259 |         selection: {
260 |           active: new vscode.Position(0, 0)
261 |         },
262 |         edit: jest.fn(() => Promise.resolve(true))
263 |       };
264 |       (vscode.window as any).activeTextEditor = mockEditor;
265 |
266 |       const contextInjectedPromise = new Promise((resolve) => {
267 |         adapter.once('contextInjected', resolve);
268 |       });
269 |
270 |       await adapter.injectContextToDocument();
```

```
271 |       await contextInjectedPromise;
272 |
273 |       expect(mockEditor.edit).toHaveBeenCalled();
274 |     });
275 |   });
276 |
277 |   describe('configuration handling', () => {
278 |     it('should respect validation triggers', async () => {
279 |       const configWithTriggers: CursorAdapterConfig = {
280 |         ...mockConfig,
281 |         validationTriggers: ['typescript']
282 |       };
283 |
284 |       const adapterWithTriggers = new CursorAdapter(configWithTriggers);
285 |       await adapterWithTriggers.initialize();
286 |
287 |       // Test private method through validation
288 |       const tsContextData: CursorContextData = {
289 |         documentUri: 'file:///test/app.ts',
290 |         content: 'const app = {};',
291 |         language: 'typescript'
292 |       };
293 |
294 |       const jsContextData: CursorContextData = {
295 |         documentUri: 'file:///test/app.js',
296 |         content: 'const app = {};',
297 |         language: 'javascript'
298 |       };
299 |
300 |       // Both should validate, but triggers only affect event handling
301 |       const tsResult = await adapterWithTriggers.validateDocument(tsContextData);
302 |       const jsResult = await adapterWithTriggers.validateDocument(jsContextData);
303 |
304 |       expect(tsResult).toBeDefined();
305 |       expect(jsResult).toBeDefined();
306 |
307 |       adapterWithTriggers.dispose();
308 |     });
309 |
310 |     it('should handle disabled real-time validation', async () => {
311 |       const configDisabled: CursorAdapterConfig = {
312 |         ...mockConfig,
313 |         enableRealTimeValidation: false
314 |       };
315 |
```

```
316 |       const adapterDisabled = new CursorAdapter(configDisabled);
317 |       await adapterDisabled.initialize();
318 |
319 |       // Event listeners should not be set up
320 |       expect(vscode.workspace.onDidChangeTextDocument).not.toHaveBeenCalled();
321 |
322 |       adapterDisabled.dispose();
323 |     });
324 |   });
325 |
326 |   describe('cache management', () => {
327 |     beforeEach(async () => {
328 |       await adapter.initialize();
329 |     });
330 |
331 |     it('should clear validation cache', () => {
332 |       // Add some items to cache first
333 |       const contextData: CursorContextData = {
334 |         documentUri: 'file:///test/api.yaml',
335 |         content: 'openapi: 3.0.0',
336 |         language: 'yaml'
337 |       };
338 |
339 |       return adapter.validateDocument(contextData).then(() => {
340 |         const statsBefore = adapter.getStatistics();
341 |         expect(statsBefore.cacheSize).toBeGreaterThan(0);
342 |
343 |         adapter.clearValidationCache();
344 |
345 |         const statsAfter = adapter.getStatistics();
346 |         expect(statsAfter.cacheSize).toBe(0);
347 |         expect(vscode.window.showInformationMessage).toHaveBeenCalledWith('Validation
cache cleared');
348 |       });
349 |     });
350 |   });
351 |
352 |   describe('statistics and monitoring', () => {
353 |     it('should provide accurate statistics', async () => {
354 |       const stats = adapter.getStatistics();
355 |
356 |       expect(stats).toHaveProperty('isActive');
357 |       expect(stats).toHaveProperty('cacheSize');
358 |       expect(stats).toHaveProperty('config');
359 |       expect(stats).toHaveProperty('disposablesCount');
```

```
360 |
361 |       expect(stats.isActive).toBe(false); // Not initialized yet
362 |       expect(stats.cacheSize).toBe(0);
363 |       expect(stats.config).toBe(mockConfig);
364 |
365 |       await adapter.initialize();
366 |
367 |       const statsAfterInit = adapter.getStatistics();
368 |       expect(statsAfterInit.isActive).toBe(true);
369 |     });
370 |   });
371 |
372 |   describe('disposal', () => {
373 |     it('should dispose resources properly', async () => {
374 |       await adapter.initialize();
375 |
376 |       const statsBefore = adapter.getStatistics();
377 |       expect(statsBefore.isActive).toBe(true);
378 |       expect(statsBefore.disposablesCount).toBeGreaterThan(0);
379 |
380 |       adapter.dispose();
381 |
382 |       const statsAfter = adapter.getStatistics();
383 |       expect(statsAfter.isActive).toBe(false);
384 |       expect(statsAfter.disposablesCount).toBe(0);
385 |       expect(statsAfter.cacheSize).toBe(0);
386 |     });
387 |   });
388 |
389 |   describe('event emission', () => {
390 |     beforeEach(async () => {
391 |       await adapter.initialize();
392 |     });
393 |
394 |     it('should emit validation results', async () => {
395 |       const contextData: CursorContextData = {
396 |         documentUri: 'file:///test/api.json',
397 |         content: '{"openapi": "3.0.0"}',
398 |         language: 'json'
399 |       };
400 |
401 |       const validationResultPromise = new
Promise<CursorValidationResult>((resolve) => {
402 |         adapter.once('validationResult', resolve);
403 |       });
404 |
405 |       // Simulate document change event
```

```
406 |       const mockEvent = {
407 |         document: {
408 |           uri: { toString: () => contextData.documentUri },
409 |           getText: () => contextData.content,
410 |           languageId: contextData.language
411 |         }
412 |       };
413 |
414 |       // Access private method through any
415 |       await (adapter as any).handleDocumentChange(mockEvent);
416 |
417 |       // Wait for debounced validation
418 |       await new Promise(resolve => setTimeout(resolve, 600));
419 |
420 |       const result = await validationResultPromise;
421 |       expect(result).toBeDefined();
422 |       expect(result.timestamp).toBeGreaterThan(0);
423 |     });
424 |   });
425 | });
426 |
```

/tests/unit/error-recovery.test.ts:

```
  1 | /**
  2 |  * Error Recovery System Unit Tests
  3 |  */
  4 |
  5 | import {
  6 |   ErrorClassifier,
  7 |   RetryManager,
  8 |   CircuitBreaker,
  9 |   StateManager,
 10 |   MetricsCollector,
 11 |   RecoveryManager,
 12 |   ErrorRecoverySystemFactory,
 13 |   ErrorRecoveryUtils,
 14 |   defaultErrorRecoverySystem,
 15 |   ErrorCategory,
 16 |   ErrorSeverity,
 17 |   RecoveryAction,
 18 |   RetryStrategy,
 19 |   CircuitBreakerState,
 20 |   createDefaultErrorClassifier,
 21 |   createRetryManager,
```

```
22 |   createCircuitBreaker,
23 |   createStateManager,
24 |   createMetricsCollector,
25 |   createRecoveryManager
26 | } from '../../src/core/error-recovery';
27 |
28 | describe('Error Recovery System', () => {
29 |   describe('ErrorClassifier', () => {
30 |     let classifier: ErrorClassifier;
31 |
32 |     beforeEach(() => {
33 |       classifier = createDefaultErrorClassifier();
34 |     });
35 |
36 |     test('should classify network errors correctly', () => {
37 |       const error = new Error('ECONNREFUSED connection failed');
38 |       const classification = classifier.classify(error);
39 |
40 |       expect(classification.category).toBe(ErrorCategory.NETWORK);
41 |       expect(classification.severity).toBe(ErrorSeverity.MEDIUM);
42 |       expect(classification.isRecoverable).toBe(true);
43 |       expect(classification.recommendedAction).toBe(RecoveryAction.RETRY);
44 |     });
45 |
46 |     test('should classify timeout errors correctly', () => {
47 |       const error = new Error('Request timed out');
48 |       const classification = classifier.classify(error);
49 |
50 |       expect(classification.category).toBe(ErrorCategory.TIMEOUT);
51 |       expect(classification.isRecoverable).toBe(true);
52 |     });
53 |
54 |     test('should classify authentication errors correctly', () => {
55 |       const error = new Error('401 Unauthorized');
56 |       const classification = classifier.classify(error);
57 |
58 |       expect(classification.category).toBe(ErrorCategory.AUTHENTICATION);
59 |       expect(classification.isRecoverable).toBe(false);
60                                                                          |
expect(classification.recommendedAction).toBe(RecoveryAction.ESCALATE);
61 |     });
62 |
63 |     test('should handle unknown errors', () => {
64 |       const error = new Error('Some unknown error');
65 |       const classification = classifier.classify(error);
66 |
67 |       expect(classification.category).toBe(ErrorCategory.UNKNOWN);
```

```
68 |       expect(classification.isRecoverable).toBe(true);
69 |     });
70 |
71 |     test('should allow custom classification rules', () => {
72 |       const customClassification = {
73 |         category: ErrorCategory.VALIDATION,
74 |         severity: ErrorSeverity.HIGH,
75 |         isRecoverable: false,
76 |         recommendedAction: RecoveryAction.ESCALATE,
77 |         metadata: {}
78 |       };
79 |
80 |       classifier.addRule(
81 |         (error) => error.message.includes('custom'),
82 |         customClassification,
83 |         200
84 |       );
85 |
86 |       const error = new Error('custom error message');
87 |       const classification = classifier.classify(error);
88 |
89 |       expect(classification.category).toBe(ErrorCategory.VALIDATION);
90 |     });
91 |   });
92 |
93 |   describe('RetryManager', () => {
94 |     let retryManager: RetryManager;
95 |
96 |     beforeEach(() => {
97 |       retryManager = createRetryManager();
98 |     });
99 |
100 |     test('should execute operation successfully on first attempt', async () =>
{
101 |       const operation = jest.fn().mockResolvedValue('success');
102 |       const config = {
103 |         strategy: RetryStrategy.FIXED,
104 |         maxAttempts: 3,
105 |         baseDelay: 100,
106 |         maxDelay: 1000,
107 |         backoffMultiplier: 2,
108 |         jitter: false,
109 |         retryableErrors: [ErrorCategory.NETWORK]
110 |       };
111 |       const context = {
112 |         operationId: 'test',
113 |         attempt: 0,
```

```
114 |         startTime: new Date(),
115 |         metadata: {},
116 |         checkpoints: []
117 |       };
118 |
119 |       const result = await retryManager.execute(operation, config, context);
120 |
121 |       expect(result).toBe('success');
122 |       expect(operation).toHaveBeenCalledTimes(1);
123 |     });
124 |
125 |     test('should retry on retryable errors', async () => {
126 |       const operation = jest.fn()
127 |         .mockRejectedValueOnce(new Error('ECONNREFUSED'))
128 |         .mockResolvedValue('success');
129 |
130 |       const config = {
131 |         strategy: RetryStrategy.FIXED,
132 |         maxAttempts: 3,
133 |         baseDelay: 10,
134 |         maxDelay: 1000,
135 |         backoffMultiplier: 2,
136 |         jitter: false,
137 |         retryableErrors: [ErrorCategory.NETWORK]
138 |       };
139 |       const context = {
140 |         operationId: 'test',
141 |         attempt: 0,
142 |         startTime: new Date(),
143 |         metadata: {},
144 |         checkpoints: []
145 |       };
146 |
147 |       const result = await retryManager.execute(operation, config, context);
148 |
149 |       expect(result).toBe('success');
150 |       expect(operation).toHaveBeenCalledTimes(2);
151 |     });
152 |
153 |     test('should not retry on non-retryable errors', async () => {
154 |       const operation = jest.fn().mockRejectedValue(new Error('401
Unauthorized'));
155 |       const config = {
156 |         strategy: RetryStrategy.FIXED,
157 |         maxAttempts: 3,
158 |         baseDelay: 10,
159 |         maxDelay: 1000,
```

```
160 |         backoffMultiplier: 2,
161 |         jitter: false,
162 |         retryableErrors: [ErrorCategory.NETWORK]
163 |       };
164 |       const context = {
165 |         operationId: 'test',
166 |         attempt: 0,
167 |         startTime: new Date(),
168 |         metadata: {},
169 |         checkpoints: []
170 |       };
171 |
172 |       await expect(retryManager.execute(operation, config, context))
173 |         .rejects.toThrow('401 Unauthorized');
174 |       expect(operation).toHaveBeenCalledTimes(1);
175 |     });
176 |
177 |     test('should calculate exponential backoff delay correctly', () => {
178 |       const config = {
179 |         strategy: RetryStrategy.EXPONENTIAL,
180 |         maxAttempts: 5,
181 |         baseDelay: 1000,
182 |         maxDelay: 30000,
183 |         backoffMultiplier: 2,
184 |         jitter: false,
185 |         retryableErrors: []
186 |       };
187 |
188 |       expect(retryManager.calculateDelay(1, config)).toBe(1000);
189 |       expect(retryManager.calculateDelay(2, config)).toBe(2000);
190 |       expect(retryManager.calculateDelay(3, config)).toBe(4000);
191 |       expect(retryManager.calculateDelay(4, config)).toBe(8000);
192 |     });
193 |   });
194 |
195 |   describe('CircuitBreaker', () => {
196 |     let circuitBreaker: CircuitBreaker;
197 |
198 |     beforeEach(() => {
199 |       circuitBreaker = createCircuitBreaker({
200 |         failureThreshold: 3,
201 |         recoveryTimeout: 60000, // 60 seconds to ensure it stays open during test
202 |         monitoringWindow: 5000,
203 |         minimumThroughput: 2
204 |       });
205 |     });
206 |
```

```
207 |     test('should start in CLOSED state', () => {
208 |       expect(circuitBreaker.getState()).toBe(CircuitBreakerState.CLOSED);
209 |     });
210 |
211 |     test('should execute operation successfully when closed', async () => {
212 |       const operation = jest.fn().mockResolvedValue('success');
213 |       const result = await circuitBreaker.execute(operation);
214 |
215 |       expect(result).toBe('success');
216 |       expect(operation).toHaveBeenCalledTimes(1);
217 |     });
218 |
219 |     test('should open circuit after failure threshold', async () => {
220 |       const operation = jest.fn().mockRejectedValue(new Error('failure'));
221 |
222 |       // Execute enough failures to open circuit
223 |       for (let i = 0; i < 5; i++) {
224 |         try {
225 |           await circuitBreaker.execute(operation);
226 |         } catch (error) {
227 |           // Expected failures
228 |         }
229 |       }
230 |
231 |       expect(circuitBreaker.getState()).toBe(CircuitBreakerState.OPEN);
232 |     });
233 |
234 |     test('should reject operations when open', async () => {
235 |       // Force circuit to open
236 |       circuitBreaker.forceOpen();
237 |
238 |       const operation = jest.fn().mockResolvedValue('success');
239 |
240 |       await expect(circuitBreaker.execute(operation))
241 |         .rejects.toThrow('Circuit breaker is OPEN');
242 |       expect(operation).not.toHaveBeenCalled();
243 |     });
244 |   });
245 |
246 |   describe('StateManager', () => {
247 |     let stateManager: StateManager;
248 |
249 |     beforeEach(() => {
250 |       stateManager = createStateManager();
251 |     });
252 |
253 |     test('should create checkpoints', () => {
```

```
254 |       const state = { data: 'test' };
255 |       const checkpoint = stateManager.createCheckpoint('op1', state, 'Test checkpoint');
256 |
257 |       expect(checkpoint.id).toBeDefined();
258 |       expect(checkpoint.description).toBe('Test checkpoint');
259 |       expect(checkpoint.timestamp).toBeInstanceOf(Date);
260 |     });
261 |
262 |     test('should retrieve checkpoints by operation', () => {
263 |       stateManager.createCheckpoint('op1', { data: 'test1' }, 'Checkpoint 1');
264 |       stateManager.createCheckpoint('op1', { data: 'test2' }, 'Checkpoint 2');
265 |       stateManager.createCheckpoint('op2', { data: 'test3' }, 'Checkpoint 3');
266 |
267 |       const op1Checkpoints = stateManager.getCheckpoints('op1');
268 |       expect(op1Checkpoints).toHaveLength(2);
269 |
270 |       const allCheckpoints = stateManager.getCheckpoints();
271 |       expect(allCheckpoints).toHaveLength(3);
272 |     });
273 |
274 |     test('should rollback to checkpoint', async () => {
275 |       const state = { data: 'test' };
276 |       const checkpoint = stateManager.createCheckpoint('op1', state, 'Test checkpoint');
277 |
278 |       const restoredState = await stateManager.rollback(checkpoint.id);
279 |       expect(restoredState).toEqual(state);
280 |     });
281 |
282 |     test('should clear checkpoints', () => {
283 |       stateManager.createCheckpoint('op1', { data: 'test1' }, 'Checkpoint 1');
284 |       stateManager.createCheckpoint('op2', { data: 'test2' }, 'Checkpoint 2');
285 |
286 |       stateManager.clearCheckpoints('op1');
287 |       expect(stateManager.getCheckpoints('op1')).toHaveLength(0);
288 |       expect(stateManager.getCheckpoints('op2')).toHaveLength(1);
289 |
290 |       stateManager.clearCheckpoints();
291 |       expect(stateManager.getCheckpoints()).toHaveLength(0);
292 |     });
293 |   });
294 |
295 |   describe('MetricsCollector', () => {
296 |     let metricsCollector: MetricsCollector;
297 |
298 |     beforeEach(() => {
```

```
299 |       metricsCollector = createMetricsCollector();
300 |     });
301 |
302 |     afterEach(() => {
303 |       metricsCollector.destroy();
304 |     });
305 |
306 |     test('should record operations', () => {
307 |       metricsCollector.recordOperation('op1', true, 100);
308 |       metricsCollector.recordOperation('op2', false, 200);
309 |
310 |       const stats = metricsCollector.getStats();
311 |       expect(stats.totalOperations).toBe(2);
312 |       expect(stats.totalFailures).toBe(1);
313 |       expect(stats.successRate).toBe(0.5);
314 |     });
315 |
316 |     test('should record errors', () => {
317 |       const error = new Error('test error');
318 |       const classification = {
319 |         category: ErrorCategory.NETWORK,
320 |         severity: ErrorSeverity.MEDIUM,
321 |         isRecoverable: true,
322 |         recommendedAction: RecoveryAction.RETRY,
323 |         metadata: {}
324 |       };
325 |
326 |       metricsCollector.recordError(error, classification);
327 |
328 |       const stats = metricsCollector.getStats();
329 |       expect(stats.errorBreakdown[ErrorCategory.NETWORK]).toBe(1);
330 |     });
331 |
332 |     test('should record recoveries', () => {
333 |       metricsCollector.recordRecovery(RecoveryAction.RETRY, true, 500);
334 |       metricsCollector.recordRecovery(RecoveryAction.FALLBACK, false, 1000);
335 |
336 |       const stats = metricsCollector.getStats();
337 |       expect(stats.totalRecoveries).toBe(2);
338 |       expect(stats.recoveryBreakdown[RecoveryAction.RETRY]).toBe(1);
339 |       expect(stats.recoveryBreakdown[RecoveryAction.FALLBACK]).toBe(1);
340 |     });
341 |   });
342 |
343 |   describe('ErrorRecoverySystemFactory', () => {
344 |     const recoverySystems: any[] = [];
345 |
```

```
346 |     afterEach(async () => {
347 |       // Clean up all recovery systems created during tests
348 |       for (const system of recoverySystems) {
349 |         if (system.recoveryManager && typeof system.recoveryManager.shutdown
=== 'function') {
350 |           await system.recoveryManager.shutdown();
351 |         }
352 |       }
353 |       recoverySystems.length = 0;
354 |     });
355 |
356 |     test('should create basic recovery system', () => {
357 |       const system = ErrorRecoverySystemFactory.createBasic();
358 |       recoverySystems.push(system);
359 |
360 |       expect(system.recoveryManager).toBeDefined();
361 |       expect(system.execute).toBeDefined();
362 |       expect(system.getStats).toBeDefined();
363 |       expect(system.getHealth).toBeDefined();
364 |     });
365 |
366 |     test('should create development recovery system', () => {
367 |       const system = ErrorRecoverySystemFactory.createForDevelopment();
368 |       recoverySystems.push(system);
369 |       expect(system.recoveryManager).toBeDefined();
370 |     });
371 |
372 |     test('should create production recovery system', () => {
373 |       const system = ErrorRecoverySystemFactory.createForProduction();
374 |       recoverySystems.push(system);
375 |       expect(system.recoveryManager).toBeDefined();
376 |     });
377 |
378 |     test('should create testing recovery system', () => {
379 |       const system = ErrorRecoverySystemFactory.createForTesting();
380 |       recoverySystems.push(system);
381 |       expect(system.recoveryManager).toBeDefined();
382 |     });
383 |   });
384 |
385 |   describe('ErrorRecoveryUtils', () => {
386 |     test('should create retry wrapper', async () => {
387 |       let attempts = 0;
388 |       const operation = jest.fn().mockImplementation(() => {
389 |         attempts++;
390 |         if (attempts < 3) {
391 |           return Promise.reject(new Error('ECONNREFUSED'));
```

```
392 |         }
393 |         return Promise.resolve('success');
394 |       });
395 |
396 |       const result = await ErrorRecoveryUtils.withRetry(operation, 3, 10);
397 |       expect(result).toBe('success');
398 |       expect(operation).toHaveBeenCalledTimes(3);
399 |     });
400 |
401 |     test('should create timeout wrapper', async () => {
402 |       const slowOperation = () => new Promise(resolve => setTimeout(resolve,
1000));
403 |
404 |       await expect(ErrorRecoveryUtils.withTimeout(slowOperation, 100))
405 |         .rejects.toThrow('Operation timed out');
406 |     });
407 |   });
408 |
409 |   afterAll(async () => {
410 |     // Clean up the default error recovery system
411   |             if   (defaultErrorRecoverySystem.recoveryManager   &&   typeof
defaultErrorRecoverySystem.recoveryManager.shutdown === 'function') {
412 |       await defaultErrorRecoverySystem.recoveryManager.shutdown();
413 |     }
414 |   });
415 | });
416 |
```

/tests/unit/html-report.test.ts:

```
  1 | import { describe, it, expect, beforeEach, afterEach, jest } from '@jest/globals';
  2 | import * as fs from 'fs/promises';
  3 | import * as path from 'path';
  4 | import { HtmlReportGenerator } from '../../src/core/html-report';
  5 | import { TemplateManager } from '../../src/core/html-report/template-manager';
  6 | import { ChartManager } from '../../src/core/html-report/chart-manager';
  7 | import type {
  8 |   HtmlReportConfig,
  9 |   ReportData,
 10 |   ValidationSummary,
 11 |   PerformanceMetrics,
 12 |   ExportOptions
 13 | } from '../../src/core/html-report/types';
 14 | import { createMockLogger } from '../helpers/mock-logger';
 15 |
```

```
16 | // Mock fs module
17 | jest.mock('fs/promises');
18 | const mockFs = fs as jest.Mocked<typeof fs>;
19 |
20 | describe('HtmlReportGenerator', () => {
21 |   let generator: HtmlReportGenerator;
22 |   let config: HtmlReportConfig;
23 |   let mockLogger: ReturnType<typeof createMockLogger>;
24 |   let tempDir: string;
25 |
26 |   beforeEach(async () => {
27 |     mockLogger = createMockLogger();
28 |     tempDir = path.join(__dirname, 'temp-reports');
29 |
30 |     config = {
31 |       outputDirectory: tempDir,
32 |       templateDirectory: path.join(tempDir, 'templates'),
33 |       defaultTheme: 'default',
34 |       chartConfig: {
35 |         colors: ['#FF6384', '#36A2EB', '#FFCE56'],
36 |         globalOptions: {},
37 |         responsive: true
38 |       },
39 |       exportConfig: {
40 |         formats: ['html', 'pdf', 'json'],
41 |         pdfOptions: {
42 |           format: 'A4',
43 |           orientation: 'portrait',
44 |           margin: { top: '1cm', right: '1cm', bottom: '1cm', left: '1cm' },
45 |           printBackground: true,
46 |           displayHeaderFooter: false
47 |         }
48 |       },
49 |       performance: {
50 |         maxReportSize: 50,
51 |         enableCaching: true,
52 |         cacheTtl: 3600
53 |       }
54 |     };
55 |
56 |     generator = new HtmlReportGenerator(config, mockLogger);
57 |
58 |     // Mock fs operations
59 |     mockFs.access.mockImplementation((path: any) => {
60 |       // Simulate template directory not existing to trigger default template
loading
61 |       if (path.includes('templates')) {
```

```
 62 |         return Promise.reject(new Error('Directory not found'));
 63 |       }
 64 |       return Promise.resolve(undefined);
 65 |     });
 66 |     mockFs.mkdir.mockResolvedValue(undefined);
 67 |     mockFs.readdir.mockResolvedValue([]);
 68 |     mockFs.readFile.mockResolvedValue('');
 69 |     mockFs.writeFile.mockResolvedValue(undefined);
 70 |   });
 71 |
 72 |   afterEach(() => {
 73 |     jest.clearAllMocks();
 74 |   });
 75 |
 76 |   describe('initialization', () => {
 77 |     it('should initialize successfully', async () => {
 78 |       await generator.initialize();
 79 |
 80 |       // Should log successful initialization
 81 |         expect(mockLogger.info).toHaveBeenCalledWith('HTML Report Generator
initialized successfully');
 82 |       // Should log default templates loading
 83 |               expect(mockLogger.info).toHaveBeenCalledWith('Loaded  default
templates');
 84 |     });
 85 |
 86 |     it('should create output directory if it does not exist', async () => {
 87 |       mockFs.access.mockRejectedValueOnce(new Error('Directory not found'));
 88 |
 89 |       await generator.initialize();
 90 |
 91 |       expect(mockFs.mkdir).toHaveBeenCalledWith(tempDir, { recursive: true });
 92 |     });
 93 |
 94 |     it('should emit initialized event', async () => {
 95 |       const initSpy = jest.fn();
 96 |       generator.on('initialized', initSpy);
 97 |
 98 |       await generator.initialize();
 99 |
100 |       expect(initSpy).toHaveBeenCalled();
101 |     });
102 |   });
103 |
104 |   describe('report generation', () => {
105 |     beforeEach(async () => {
106 |       await generator.initialize();
```

```
107 |     });
108 |
109 |     it('should generate basic HTML report', async () => {
110 |       const reportData: ReportData = {
111 |         metadata: {
112 |           title: 'Test Report',
113 |           description: 'Test report description',
114 |           generatedAt: new Date('2024-01-01T00:00:00Z'),
115 |           version: '1.0.0',
116 |           author: 'Test Author'
117 |         },
118 |         sections: [
119 |           {
120 |             id: 'test-section',
121 |             title: 'Test Section',
122 |             content: '<p>Test content</p>',
123 |             charts: []
124 |           }
125 |         ]
126 |       };
127 |
128 |       const result = await generator.generateReport(reportData);
129 |
130 |       expect(result).toContain('Test Report');
131 |       expect(result).toContain('Test report description');
132 |       expect(result).toContain('Test Section');
133 |       expect(result).toContain('<p>Test content</p>');
134 |     });
135 |
136 |     it('should generate validation summary report', async () => {
137 |       const validationResults: ValidationSummary[] = [
138 |         {
139 |           specId: 'spec1',
140 |           specName: 'API Spec 1',
141 |           status: 'passed',
142 |           totalValidations: 10,
143 |           passedValidations: 10,
144 |           failedValidations: 0,
145 |           warnings: 0,
146 |           timestamp: new Date(),
147 |           duration: 100,
148 |           errors: [],
149 |           coverage: {
150 |             endpointCoverage: 100,
151 |             schemaCoverage: 95,
152 |             parameterCoverage: 90,
153 |             responseCoverage: 85,
```

```
154 |             overallCoverage: 92.5
155 |           }
156 |         },
157 |         {
158 |           specId: 'spec2',
159 |           specName: 'API Spec 2',
160 |           status: 'failed',
161 |           totalValidations: 8,
162 |           passedValidations: 6,
163 |           failedValidations: 2,
164 |           warnings: 1,
165 |           timestamp: new Date(),
166 |           duration: 150,
167 |           errors: [
168 |             {
169 |               code: 'E001',
170 |               message: 'Invalid schema',
171 |               severity: 'error',
172 |               location: { path: '/paths/users', line: 10 },
173 |               rule: 'schema-validation'
174 |             }
175 |           ],
176 |           coverage: {
177 |             endpointCoverage: 80,
178 |             schemaCoverage: 75,
179 |             parameterCoverage: 70,
180 |             responseCoverage: 65,
181 |             overallCoverage: 72.5
182 |           }
183 |         }
184 |       ];
185 |
186 |       const result = await generator.generateValidationSummary(validationResults);
187 |
188 |       expect(result).toContain('API Validation Summary');
189 |       expect(result).toContain('API Spec 1');
190 |       expect(result).toContain('API Spec 2');
191 |       expect(result).toContain('passed');
192 |       expect(result).toContain('failed');
193 |     });
194 |
195 |     it('should generate performance report', async () => {
196 |       const performanceMetrics: PerformanceMetrics[] = [
197 |         {
198 |           id: 'metric1',
199 |           name: 'Response Time',
```

```
200 |           value: 150,
201 |           unit: 'ms',
202 |           timestamp: new Date(),
203 |           category: 'response_time'
204 |         },
205 |         {
206 |           id: 'metric2',
207 |           name: 'Throughput',
208 |           value: 1000,
209 |           unit: 'req/s',
210 |           timestamp: new Date(),
211 |           category: 'throughput'
212 |         }
213 |       ];
214 |
215 |       const result = await generator.generatePerformanceReport(performanceMetrics);
216 |
217 |       expect(result).toContain('Performance Metrics Report');
218 |       expect(result).toContain('Response Time');
219 |       expect(result).toContain('Throughput');
220 |     });
221 |
222 |     it('should include charts when enabled', async () => {
223 |       const reportData: ReportData = {
224 |         metadata: {
225 |           title: 'Chart Test Report',
226 |           description: 'Report with charts',
227 |           generatedAt: new Date(),
228 |           version: '1.0.0',
229 |           author: 'Test'
230 |         },
231 |         charts: [
232 |           {
233 |             id: 'test-chart',
234 |             type: 'pie',
235 |             title: 'Test Chart',
236 |             containerId: 'test-chart',
237 |             data: {
238 |               labels: ['A', 'B', 'C'],
239 |               datasets: [{
240 |                 label: 'Test Data',
241 |                 data: [10, 20, 30],
242 |                 backgroundColor: ['#FF6384', '#36A2EB', '#FFCE56']
243 |               }]
244 |             },
245 |             options: {
```

```
246 |               responsive: true,
247 |               maintainAspectRatio: false,
248 |               plugins: {
249 |                 legend: { display: true, position: 'bottom' }
250 |               }
251 |             }
252 |           }
253 |         ],
254 |         sections: []
255 |       };
256 |
257 |       const result = await generator.generateReport(reportData, { includeCharts:
true });
258 |
259 |       expect(result).toContain('chart.js');
260 |       expect(result).toContain('test-chart');
261 |       expect(result).toContain('new Chart');
262 |     });
263 |
264 |     it('should save report to file when output path is provided', async () =>
{
265 |       const reportData: ReportData = {
266 |         metadata: {
267 |           title: 'Save Test',
268 |           description: 'Test saving',
269 |           generatedAt: new Date(),
270 |           version: '1.0.0',
271 |           author: 'Test'
272 |         },
273 |         sections: []
274 |       };
275 |
276 |       const outputPath = 'test-report.html';
277 |       await generator.generateReport(reportData, { outputPath });
278 |
279 |       expect(mockFs.writeFile).toHaveBeenCalledWith(
280 |         path.resolve(tempDir, outputPath),
281 |         expect.any(String),
282 |         'utf-8'
283 |       );
284 |     });
285 |
286 |     it('should emit reportGenerated event', async () => {
287 |       const reportSpy = jest.fn();
288 |       generator.on('reportGenerated', reportSpy);
289 |
290 |       const reportData: ReportData = {
```

```
291 |         metadata: {
292 |           title: 'Event Test',
293 |           description: 'Test events',
294 |           generatedAt: new Date(),
295 |           version: '1.0.0',
296 |           author: 'Test'
297 |         },
298 |         sections: []
299 |       };
300 |
301 |       await generator.generateReport(reportData);
302 |
303 |       expect(reportSpy).toHaveBeenCalledWith(
304 |         expect.objectContaining({
305 |           data: reportData,
306 |           content: expect.any(String)
307 |         })
308 |       );
309 |     });
310 |   });
311 |
312 |   describe('export functionality', () => {
313 |     beforeEach(async () => {
314 |       await generator.initialize();
315 |     });
316 |
317 |     it('should export as HTML', async () => {
318 |       const htmlContent = '<html><body>Test</body></html>';
319 |       const outputPath = 'test.html';
320 |
321 |       await generator.exportReport(htmlContent, 'html', outputPath);
322 |
323 |       expect(mockFs.writeFile).toHaveBeenCalledWith(outputPath, htmlContent,
'utf-8');
324 |     });
325 |
326 |     it('should export as JSON', async () => {
327 |       const htmlContent = '<html><body>Test</body></html>';
328 |       const outputPath = 'test.json';
329 |
330 |       await generator.exportReport(htmlContent, 'json', outputPath);
331 |
332 |       expect(mockFs.writeFile).toHaveBeenCalledWith(
333 |         outputPath,
334 |         expect.stringContaining('"exportType": "html-report"'),
335 |         'utf-8'
336 |       );
```

```
337 |     });
338 |
339 |     it('should export as PDF (placeholder)', async () => {
340 |       const htmlContent = '<html><body>Test</body></html>';
341 |       const outputPath = 'test.pdf';
342 |
343 |       await generator.exportReport(htmlContent, 'pdf', outputPath);
344 |
345 |       expect(mockFs.writeFile).toHaveBeenCalledWith(
346 |         outputPath,
347 |         expect.stringContaining('PDF Export Placeholder'),
348 |         'utf-8'
349 |       );
350 |       expect(mockLogger.warn).toHaveBeenCalledWith(
351 |         expect.stringContaining('PDF export is a placeholder')
352 |       );
353 |     });
354 |
355 |     it('should emit reportExported event', async () => {
356 |       const exportSpy = jest.fn();
357 |       generator.on('reportExported', exportSpy);
358 |
359 |       const htmlContent = '<html><body>Test</body></html>';
360 |       await generator.exportReport(htmlContent, 'html', 'test.html');
361 |
362 |       expect(exportSpy).toHaveBeenCalledWith(
363 |         expect.objectContaining({
364 |           format: 'html',
365 |           outputPath: 'test.html'
366 |         })
367 |       );
368 |     });
369 |
370 |     it('should throw error for unsupported format', async () => {
371 |       const htmlContent = '<html><body>Test</body></html>';
372 |
373 |       await expect(
374 |         generator.exportReport(htmlContent, 'xml' as any, 'test.xml')
375 |       ).rejects.toThrow('Unsupported export format: xml');
376 |     });
377 |   });
378 |
379 |   describe('template management', () => {
380 |     beforeEach(async () => {
381 |       await generator.initialize();
382 |     });
383 |
```

```
384 |     it('should get available templates', () => {
385 |       const templates = generator.getAvailableTemplates();
386 |       expect(Array.isArray(templates)).toBe(true);
387 |     });
388 |
389 |     it('should add custom template', async () => {
390 |       const templateContent = '<div>{{title}}</div>';
391 |
392 |       await generator.addCustomTemplate('custom', templateContent);
393 |
394 |       const templates = generator.getAvailableTemplates();
395 |       expect(templates).toContain('custom');
396 |     });
397 |
398 |     it('should get report statistics', () => {
399 |       const stats = generator.getReportStatistics();
400 |
401 |       expect(stats).toHaveProperty('templatesLoaded');
402 |       expect(stats).toHaveProperty('reportsGenerated');
403 |       expect(typeof stats.templatesLoaded).toBe('number');
404 |     });
405 |   });
406 |
407 |   describe('error handling', () => {
408 |     it('should throw error when not initialized', async () => {
409 |       const uninitializedGenerator = new HtmlReportGenerator(config,
mockLogger);
410 |
411 |       const reportData: ReportData = {
412 |         metadata: {
413 |           title: 'Test',
414 |           description: 'Test',
415 |           generatedAt: new Date(),
416 |           version: '1.0.0',
417 |           author: 'Test'
418 |         },
419 |         sections: []
420 |       };
421 |
422 |       await expect(
423 |         uninitializedGenerator.generateReport(reportData)
424 |       ).rejects.toThrow('HTML Report Generator not initialized');
425 |     });
426 |
427 |     it('should handle template compilation errors', async () => {
428 |       await generator.initialize();
429 |
```

```
430 |       // Mock Handlebars.compile to throw an error
431 |       const originalCompile = require('handlebars').compile;
432 |       require('handlebars').compile = jest.fn().mockImplementation(() => {
433 |         throw new Error('Template compilation failed');
434 |       });
435 |
436 |       const invalidTemplate = '<div>{{test}}</div>';
437 |
438 |       await expect(
439 |         generator.addCustomTemplate('invalid', invalidTemplate)
440 |       ).rejects.toThrow('Template compilation failed');
441 |
442 |       // Restore original compile function
443 |       require('handlebars').compile = originalCompile;
444 |     });
445 |
446 |     it('should handle file system errors gracefully', async () => {
447 |       // Create a new generator to test initialization failure
448 |       const failingGenerator = new HtmlReportGenerator(config, mockLogger);
449 |
450 |       // Mock access to succeed for output directory but mkdir to fail
451 |       mockFs.access.mockImplementation((path: any) => {
452 |         if (path.includes('temp-reports') && !path.includes('templates')) {
453 |           return Promise.reject(new Error('Directory not found'));
454 |         }
455 |         if (path.includes('templates')) {
456 |           return Promise.reject(new Error('Directory not found'));
457 |         }
458 |         return Promise.resolve(undefined);
459 |       });
460 |
461 |       // Mock mkdir to fail for the output directory
462 |       mockFs.mkdir.mockRejectedValueOnce(new Error('Permission denied'));
463 |
464 |       await expect(failingGenerator.initialize()).rejects.toThrow('Permission
denied');
465 |     });
466 |   });
467 | });
468 |
```

/tests/unit/mcp-server.test.ts:

```
  1 | /**
  2 |  * MCP Server Tests
```

```
 3 |  *
 4 |  * Comprehensive test suite for the MCP Protocol Server implementation
 5 |  * Tests cover HTTP endpoints, WebSocket connections, JSON-RPC message handling,
 6 |  * session management, tool execution, and error handling
 7 |  */
 8 |
 9 | import { describe, test, expect, beforeEach, afterEach, jest } from '@jest/globals';
10 | import request from 'supertest';
11 | import WebSocket from 'ws';
12 | import { MCPServer, MCPServerConfig } from '../../src/core/mcp-server/index';
13 |
14 | describe('MCP Server', () => {
15 |   let server: MCPServer;
16 |   let config: MCPServerConfig;
17 |
18 |   beforeEach(() => {
19 |     config = {
20 |       name: 'test-mcp-server',
21 |       version: '1.0.0',
22 |       port: 3001,
23 |       enableCors: true,
24 |       enableWebSocket: true,
25 |       sessionTimeout: 60000,
26 |       maxSessions: 10
27 |     };
28 |     server = new MCPServer(config);
29 |   });
30 |
31 |   afterEach(async () => {
32 |     if (server) {
33 |       await server.stop();
34 |     }
35 |   });
36 |
37 |   describe('Server Lifecycle', () => {
38 |     test('should start and stop server successfully', async () => {
39 |       await server.start();
40 |       expect(server.getStatus().isRunning).toBe(true);
41 |
42 |       await server.stop();
43 |       expect(server.getStatus().isRunning).toBe(false);
44 |     });
45 |
46 |     test('should not start server twice', async () => {
47 |       await server.start();
48 |       await expect(server.start()).rejects.toThrow('Server is already
```

```
running');
 49 |     });
 50 |
 51 |     test('should handle stop when not running', async () => {
 52 |       await expect(server.stop()).resolves.not.toThrow();
 53 |     });
 54 |   });
 55 |
 56 |   describe('HTTP Endpoints', () => {
 57 |     beforeEach(async () => {
 58 |       await server.start();
 59 |     });
 60 |
 61 |     test('should respond to health check', async () => {
 62 |       const response = await request(`http://localhost:${config.port}`)
 63 |         .get('/health')
 64 |         .expect(200);
 65 |
 66 |       expect(response.body).toMatchObject({
 67 |         status: 'healthy',
 68 |         server: config.name,
 69 |         version: config.version
 70 |       });
 71 |     });
 72 |
 73 |     test('should respond to server info', async () => {
 74 |       const response = await request(`http://localhost:${config.port}`)
 75 |         .get('/info')
 76 |         .expect(200);
 77 |
 78 |       expect(response.body).toMatchObject({
 79 |         name: config.name,
 80 |         version: config.version,
 81 |         protocolVersion: '2024-11-05'
 82 |       });
 83 |     });
 84 |
 85 |     test('should handle initialize request', async () => {
 86 |       const initRequest = {
 87 |         jsonrpc: '2.0',
 88 |         id: 1,
 89 |         method: 'initialize',
 90 |         params: {
 91 |           protocolVersion: '2024-11-05',
 92 |           capabilities: {
 93 |             tools: {}
 94 |           },
```

```
 95 |           clientInfo: {
 96 |             name: 'test-client',
 97 |             version: '1.0.0'
 98 |           }
 99 |         }
100 |       };
101 |
102 |       const response = await request(`http://localhost:${config.port}`)
103 |         .post('/mcp')
104 |         .send(initRequest)
105 |         .expect(200);
106 |
107 |       expect(response.body).toMatchObject({
108 |         jsonrpc: '2.0',
109 |         id: 1,
110 |         result: {
111 |           protocolVersion: '2024-11-05',
112 |           serverInfo: {
113 |             name: config.name,
114 |             version: config.version
115 |           }
116 |         }
117 |       });
118 |
119 |       expect(response.headers['mcp-session-id']).toBeDefined();
120 |     });
121 |
122 |     test('should reject request without session ID', async () => {
123 |       const request_data = {
124 |         jsonrpc: '2.0',
125 |         id: 1,
126 |         method: 'tools/list'
127 |       };
128 |
129 |       const response = await request(`http://localhost:${config.port}`)
130 |         .post('/mcp')
131 |         .send(request_data)
132 |         .expect(400);
133 |
134 |         expect(response.body.error.message).toContain('No valid session ID
provided');
135 |     });
136 |
137 |     test('should handle tools/list request', async () => {
138 |       // First initialize
139 |       const initRequest = {
140 |         jsonrpc: '2.0',
```

```
141 |         id: 1,
142 |         method: 'initialize',
143 |         params: {}
144 |       };
145 |
146 |       const initResponse = await request(`http://localhost:${config.port}`)
147 |         .post('/mcp')
148 |         .send(initRequest);
149 |
150 |       const sessionId = initResponse.headers['mcp-session-id'];
151 |
152 |       // Then list tools
153 |       const toolsRequest = {
154 |         jsonrpc: '2.0',
155 |         id: 2,
156 |         method: 'tools/list'
157 |       };
158 |
159 |       const response = await request(`http://localhost:${config.port}`)
160 |         .post('/mcp')
161 |         .set('mcp-session-id', sessionId)
162 |         .send(toolsRequest)
163 |         .expect(200);
164 |
165 |       expect(response.body).toMatchObject({
166 |         jsonrpc: '2.0',
167 |         id: 2,
168 |         result: {
169 |           tools: expect.any(Array)
170 |         }
171 |       });
172 |
173 |       expect(response.body.result.tools.length).toBeGreaterThan(0);
174 |     });
175 |
176 |     test('should handle tools/call request', async () => {
177 |       // Initialize session
178 |       const initRequest = {
179 |         jsonrpc: '2.0',
180 |         id: 1,
181 |         method: 'initialize',
182 |         params: {}
183 |       };
184 |
185 |       const initResponse = await request(`http://localhost:${config.port}`)
186 |         .post('/mcp')
187 |         .send(initRequest);
```

```
188 |
189 |       const sessionId = initResponse.headers['mcp-session-id'];
190 |
191 |       // Call validation tool
192 |       const callRequest = {
193 |         jsonrpc: '2.0',
194 |         id: 2,
195 |         method: 'tools/call',
196 |         params: {
197 |           name: 'validate-interface',
198 |           arguments: {
199 |             frontendCode: 'const api = { getData: () => Promise<string> }',
200 |             backendCode: 'app.get("/data", (req, res) => res.json("test"))',
201 |             language: 'typescript'
202 |           }
203 |         }
204 |       };
205 |
206 |       const response = await request(`http://localhost:${config.port}`)
207 |         .post('/mcp')
208 |         .set('mcp-session-id', sessionId)
209 |         .send(callRequest)
210 |         .expect(200);
211 |
212 |       expect(response.body).toMatchObject({
213 |         jsonrpc: '2.0',
214 |         id: 2,
215 |         result: {
216 |           content: expect.any(Array)
217 |         }
218 |       });
219 |     });
220 |
221 |     test('should handle resources/list request', async () => {
222 |       // Initialize session
223 |       const initRequest = {
224 |         jsonrpc: '2.0',
225 |         id: 1,
226 |         method: 'initialize',
227 |         params: {}
228 |       };
229 |
230 |       const initResponse = await request(`http://localhost:${config.port}`)
231 |         .post('/mcp')
232 |         .send(initRequest);
233 |
234 |       const sessionId = initResponse.headers['mcp-session-id'];
```

```
235 |
236 |       // List resources
237 |       const resourcesRequest = {
238 |         jsonrpc: '2.0',
239 |         id: 2,
240 |         method: 'resources/list'
241 |       };
242 |
243 |       const response = await request(`http://localhost:${config.port}`)
244 |         .post('/mcp')
245 |         .set('mcp-session-id', sessionId)
246 |         .send(resourcesRequest)
247 |         .expect(200);
248 |
249 |       expect(response.body).toMatchObject({
250 |         jsonrpc: '2.0',
251 |         id: 2,
252 |         result: {
253 |           resources: expect.any(Array)
254 |         }
255 |       });
256 |     });
257 |
258 |     test('should handle unknown method', async () => {
259 |       // Initialize session
260 |       const initRequest = {
261 |         jsonrpc: '2.0',
262 |         id: 1,
263 |         method: 'initialize',
264 |         params: {}
265 |       };
266 |
267 |       const initResponse = await request(`http://localhost:${config.port}`)
268 |         .post('/mcp')
269 |         .send(initRequest);
270 |
271 |       const sessionId = initResponse.headers['mcp-session-id'];
272 |
273 |       // Call unknown method
274 |       const unknownRequest = {
275 |         jsonrpc: '2.0',
276 |         id: 2,
277 |         method: 'unknown/method'
278 |       };
279 |
280 |       const response = await request(`http://localhost:${config.port}`)
281 |         .post('/mcp')
```

```
282 |         .set('mcp-session-id', sessionId)
283 |         .send(unknownRequest)
284 |         .expect(200);
285 |
286 |       expect(response.body).toMatchObject({
287 |         jsonrpc: '2.0',
288 |         id: 2,
289 |         error: {
290 |           code: -32601,
291 |           message: 'Method not found: unknown/method'
292 |         }
293 |       });
294 |     });
295 |
296 |     test('should handle session termination', async () => {
297 |       // Initialize session
298 |       const initRequest = {
299 |         jsonrpc: '2.0',
300 |         id: 1,
301 |         method: 'initialize',
302 |         params: {}
303 |       };
304 |
305 |       const initResponse = await request(`http://localhost:${config.port}`)
306 |         .post('/mcp')
307 |         .send(initRequest);
308 |
309 |       const sessionId = initResponse.headers['mcp-session-id'];
310 |
311 |       // Terminate session
312 |       const response = await request(`http://localhost:${config.port}`)
313 |         .delete('/mcp')
314 |         .set('mcp-session-id', sessionId)
315 |         .expect(200);
316 |
317 |       expect(response.body).toMatchObject({
318 |         jsonrpc: '2.0',
319 |         result: {
320 |           message: 'Session terminated successfully'
321 |         }
322 |       });
323 |     });
324 |   });
325 |
326 |   describe('WebSocket Support', () => {
327 |     beforeEach(async () => {
328 |       await server.start();
```

```
329 |     });
330 |
331 |     test('should accept WebSocket connections', (done) => {
332 |       const ws = new WebSocket(`ws://localhost:${config.port}/ws`);
333 |
334 |       ws.on('open', () => {
335 |         expect(ws.readyState).toBe(WebSocket.OPEN);
336 |         ws.close();
337 |         done();
338 |       });
339 |
340 |       ws.on('error', (error) => {
341 |         done(error);
342 |       });
343 |     });
344 |
345 |     test('should handle WebSocket JSON-RPC messages', (done) => {
346 |       const ws = new WebSocket(`ws://localhost:${config.port}/ws`);
347 |
348 |       ws.on('open', () => {
349 |         const initRequest = {
350 |           jsonrpc: '2.0',
351 |           id: 1,
352 |           method: 'initialize',
353 |           params: {
354 |             clientInfo: {
355 |               name: 'websocket-client',
356 |               version: '1.0.0'
357 |             }
358 |           }
359 |         };
360 |
361 |         ws.send(JSON.stringify(initRequest));
362 |       });
363 |
364 |       ws.on('message', (data) => {
365 |         const response = JSON.parse(data.toString());
366 |
367 |         expect(response).toMatchObject({
368 |           jsonrpc: '2.0',
369 |           id: 1,
370 |           result: {
371 |             protocolVersion: '2024-11-05',
372 |             serverInfo: {
373 |               name: config.name,
374 |               version: config.version
375 |             }
```

```
376 |           }
377 |         });
378 |
379 |         ws.close();
380 |         done();
381 |       });
382 |
383 |       ws.on('error', (error) => {
384 |         done(error);
385 |       });
386 |     });
387 |   });
388 |
389 |   describe('Session Management', () => {
390 |     beforeEach(async () => {
391 |       await server.start();
392 |     });
393 |
394 |     test('should track active sessions', async () => {
395 |       const initialSessions = server.getSessions().length;
396 |
397 |       // Create a session
398 |       const initRequest = {
399 |         jsonrpc: '2.0',
400 |         id: 1,
401 |         method: 'initialize',
402 |         params: {}
403 |       };
404 |
405 |       await request(`http://localhost:${config.port}`)
406 |         .post('/mcp')
407 |         .send(initRequest);
408 |
409 |       expect(server.getSessions().length).toBe(initialSessions + 1);
410 |     });
411 |
412 |     test('should provide server status', () => {
413 |       const status = server.getStatus();
414 |
415 |       expect(status).toMatchObject({
416 |         name: config.name,
417 |         version: config.version,
418 |         isRunning: true,
419 |         port: config.port,
420 |         sessions: expect.any(Number),
421 |         tools: expect.any(Number),
422 |         resources: expect.any(Number)
```

```
423 |       });
424 |     });
425 |   });
426 |
427 |   describe('Tool and Resource Registration', () => {
428 |     test('should register custom tools', async () => {
429 |       const customTool = {
430 |         name: 'custom-tool',
431 |         description: 'A custom test tool',
432 |         inputSchema: {
433 |           type: 'object' as const,
434 |           properties: {
435 |             input: { type: 'string' }
436 |           },
437 |           required: ['input']
438 |         }
439 |       };
440 |
441 |       // Start server to register default tools
442 |       await server.start();
443 |       server.registerTool(customTool);
444 |       const status = server.getStatus();
445 |       expect(status.tools).toBeGreaterThan(2); // Default tools + custom tool
446 |     });
447 |
448 |     test('should register custom resources', async () => {
449 |       const customResource = {
450 |         uri: 'test://custom-resource',
451 |         name: 'Custom Resource',
452 |         description: 'A custom test resource',
453 |         mimeType: 'text/plain'
454 |       };
455 |
456 |       // Start server to register default resources
457 |       await server.start();
458 |       server.registerResource(customResource);
459 |       const status = server.getStatus();
460 |         expect(status.resources).toBeGreaterThan(2); // Default resources +
custom resource
461 |     });
462 |   });
463 | });
464 |
```

/tests/unit/parser.test.ts:

```
/**
 * Unit tests for Enhanced OpenAPI/Swagger Parser
 */

import { SpecParser, type ParsedSpec, type ConversionOptions } from '../../src/core/parser/index';
import { Logger } from '../../src/utils/logger/index';
import type { ValidationConfig } from '../../src/utils/config/index';

describe('SpecParser', () => {
  let parser: SpecParser;
  let mockLogger: Logger;
  let mockConfig: ValidationConfig;

  beforeEach(() => {
    mockLogger = {
      info: jest.fn(),
      debug: jest.fn(),
      warn: jest.fn(),
      error: jest.fn()
    } as any;

    mockConfig = {
      allowAdditionalProperties: true,
      strictMode: false,
      customRules: []
    } as ValidationConfig;

    parser = new SpecParser(mockConfig, mockLogger);
  });

  describe('parseSpecification', () => {
    it('should parse a valid OpenAPI 3.0 specification', async () => {
      const openApiSpec = {
        openapi: '3.0.0',
        info: {
          title: 'Test API',
          version: '1.0.0',
          description: 'A test API'
        },
        servers: [
          {
            url: 'https://api.example.com',
            description: 'Production server'
          }
        ],
```

```
        paths: {
          '/users': {
            get: {
              operationId: 'getUsers',
              summary: 'Get all users',
              responses: {
                '200': {
                  description: 'Success',
                  content: {
                    'application/json': {
                      schema: {
                        type: 'array',
                        items: { $ref: '#/components/schemas/User' }
                      }
                    }
                  }
                }
              }
            },
            post: {
              operationId: 'createUser',
              summary: 'Create a user',
              requestBody: {
                required: true,
                content: {
                  'application/json': {
                    schema: { $ref: '#/components/schemas/User' }
                  }
                }
              },
              responses: {
                '201': {
                  description: 'Created',
                  content: {
                    'application/json': {
                      schema: { $ref: '#/components/schemas/User' }
                    }
                  }
                }
              }
            }
          },
          '/users/{userId}': {
            parameters: [
              {
                name: 'userId',
                in: 'path',
```

```
                required: true,
                schema: { type: 'string' }
              }
            ],
            get: {
              operationId: 'getUserById',
              summary: 'Get user by ID',
              responses: {
                '200': {
                  description: 'Success',
                  content: {
                    'application/json': {
                      schema: { $ref: '#/components/schemas/User' }
                    }
                  }
                },
                '404': {
                  description: 'User not found'
                }
              }
            }
          },
          components: {
            schemas: {
              User: {
                type: 'object',
                required: ['id', 'name'],
                properties: {
                  id: { type: 'string' },
                  name: { type: 'string' },
                  email: { type: 'string', format: 'email' }
                }
              }
            }
          }
        };

        const result = await parser.parseSpecification(openApiSpec);

        expect(result.format).toBe('openapi');
        expect(result.version).toBe('3.0.0');
        expect(result.paths).toEqual(['/users', '/users/{userId}']);
        expect(result.operations).toHaveLength(3);
        expect(result.schemas).toHaveLength(1);
        expect(result.metadata.title).toBe('Test API');
        expect(result.metadata.servers).toHaveLength(1);
```

```
140                                                                                 |
expect(result.metadata.servers![0].url).toBe('https://api.example.com');
141 |
142 |       // Check operations
143 |         const getUsersOp = result.operations.find(op => op.operationId ===
'getUsers');
144 |       expect(getUsersOp).toBeDefined();
145 |       expect(getUsersOp!.method).toBe('GET');
146 |       expect(getUsersOp!.path).toBe('/users');
147 |
148 |        const createUserOp = result.operations.find(op => op.operationId ===
'createUser');
149 |       expect(createUserOp).toBeDefined();
150 |       expect(createUserOp!.requestBody).toBeDefined();
151 |       expect(createUserOp!.requestBody!.required).toBe(true);
152 |
153 |        const getUserByIdOp = result.operations.find(op => op.operationId ===
'getUserById');
154 |       expect(getUserByIdOp).toBeDefined();
155 |       expect(getUserByIdOp!.parameters).toHaveLength(1);
156 |       expect(getUserByIdOp!.parameters![0].name).toBe('userId');
157 |       expect(getUserByIdOp!.parameters![0].in).toBe('path');
158 |       expect(getUserByIdOp!.parameters![0].required).toBe(true);
159 |
160 |       // Check schemas
161 |       expect(result.schemas[0].name).toBe('User');
162 |       expect(result.schemas[0].type).toBe('component');
163 |     });
164 |
165 |     it('should parse a valid Swagger 2.0 specification', async () => {
166 |       const swaggerSpec = {
167 |         swagger: '2.0',
168 |         info: {
169 |           title: 'Test API',
170 |           version: '1.0.0'
171 |         },
172 |         host: 'api.example.com',
173 |         basePath: '/v1',
174 |         schemes: ['https'],
175 |         paths: {
176 |           '/users': {
177 |             get: {
178 |               operationId: 'getUsers',
179 |               summary: 'Get all users',
180 |               responses: {
181 |                 '200': {
182 |                   description: 'Success',
```

```
183 |                   schema: {
184 |                     type: 'array',
185 |                     items: { $ref: '#/definitions/User' }
186 |                   }
187 |                 }
188 |               }
189 |             }
190 |           }
191 |         },
192 |         definitions: {
193 |           User: {
194 |             type: 'object',
195 |             required: ['id', 'name'],
196 |             properties: {
197 |               id: { type: 'string' },
198 |               name: { type: 'string' }
199 |             }
200 |           }
201 |         }
202 |       };
203 |
204 |       const result = await parser.parseSpecification(swaggerSpec);
205 |
206 |       expect(result.format).toBe('swagger');
207 |       expect(result.version).toBe('2.0');
208 |       expect(result.paths).toEqual(['/users']);
209 |       expect(result.operations).toHaveLength(1);
210 |       expect(result.schemas).toHaveLength(1);
211 |       expect(result.metadata.servers).toHaveLength(1);
212                                                                                 |
expect(result.metadata.servers![0].url).toBe('https://api.example.com/v1');
213 |     });
214 |
215 |     it('should throw error for invalid specification', async () => {
216 |       const invalidSpec = {
217 |         info: {
218 |           title: 'Invalid API'
219 |         }
220 |         // Missing openapi or swagger field
221 |       };
222 |
223 |       await expect(parser.parseSpecification(invalidSpec))
224 |         .rejects.toThrow('Invalid specification: missing openapi or swagger
field');
225 |     });
226 |   });
227 |
```

```
  describe('convertSwaggerToOpenAPI', () => {
    it('should convert Swagger 2.0 to OpenAPI 3.0', async () => {
      const swaggerSpec = {
        swagger: '2.0',
        info: {
          title: 'Test API',
          version: '1.0.0',
          description: 'A test API'
        },
        host: 'api.example.com',
        basePath: '/v1',
        schemes: ['https', 'http'],
        paths: {
          '/users': {
            get: {
              operationId: 'getUsers',
              summary: 'Get all users',
              parameters: [
                {
                  name: 'limit',
                  in: 'query',
                  type: 'integer',
                  description: 'Number of users to return'
                }
              ],
              responses: {
                '200': {
                  description: 'Success',
                  schema: {
                    type: 'array',
                    items: { $ref: '#/definitions/User' }
                  }
                }
              }
            },
            post: {
              operationId: 'createUser',
              summary: 'Create a user',
              parameters: [
                {
                  name: 'user',
                  in: 'body',
                  required: true,
                  schema: { $ref: '#/definitions/User' }
                }
              ],
              responses: {
```

```
275 |                 '201': {
276 |                   description: 'Created',
277 |                   schema: { $ref: '#/definitions/User' }
278 |                 }
279 |               }
280 |             }
281 |           }
282 |         },
283 |         definitions: {
284 |           User: {
285 |             type: 'object',
286 |             required: ['id', 'name'],
287 |             properties: {
288 |               id: { type: 'string' },
289 |               name: { type: 'string' }
290 |             }
291 |           }
292 |         },
293 |         securityDefinitions: {
294 |           apiKey: {
295 |             type: 'apiKey',
296 |             name: 'X-API-Key',
297 |             in: 'header'
298 |           },
299 |           oauth2: {
300 |             type: 'oauth2',
301 |             flow: 'accessCode',
302 |             authorizationUrl: 'https://auth.example.com/oauth/authorize',
303 |             tokenUrl: 'https://auth.example.com/oauth/token',
304 |             scopes: {
305 |               'read:users': 'Read users',
306 |               'write:users': 'Write users'
307 |             }
308 |           }
309 |         }
310 |       };
311 |
312 |       const options: ConversionOptions = {
313 |         targetVersion: '3.0',
314 |         validateResult: false
315 |       };
316 |
317 |       const result = await parser.convertSwaggerToOpenAPI(swaggerSpec,
options);
318 |
319 |       expect(result.openapi).toBe('3.0');
320 |       expect(result.info.title).toBe('Test API');
```

```
321 |       expect(result.servers).toHaveLength(2);
322 |       expect(result.servers![0].url).toBe('https://api.example.com/v1');
323 |       expect(result.servers![1].url).toBe('http://api.example.com/v1');
324 |
325 |       // Check converted paths
326 |       expect(result.paths['/users'].get).toBeDefined();
327 |       expect(result.paths['/users'].post).toBeDefined();
328 |
329 |       // Check parameter conversion
330 |       const getOperation = result.paths['/users'].get;
331 |       expect(getOperation.parameters).toHaveLength(1);
332 |       expect(getOperation.parameters[0].name).toBe('limit');
333 |       expect(getOperation.parameters[0].schema.type).toBe('integer');
334 |
335 |       // Check request body conversion
336 |       const postOperation = result.paths['/users'].post;
337 |       expect(postOperation.requestBody).toBeDefined();
338 |       expect(postOperation.requestBody.required).toBe(true);
339 |
expect(postOperation.requestBody.content['application/json'].schema.$ref).toBe('#/
definitions/User');
340 |
341 |       // Check response conversion
342 |
expect(postOperation.responses['201'].content['application/json'].schema.$ref).toB
e('#/definitions/User');
343 |
344 |       // Check components
345 |       expect(result.components.schemas.User).toBeDefined();
346 |       expect(result.components.securitySchemes.apiKey).toBeDefined();
347 |       expect(result.components.securitySchemes.oauth2).toBeDefined();
348 |
expect(result.components.securitySchemes.oauth2.flows.authorizationCode).toBeDefin
ed();
349 |     });
350 |
351 |     it('should throw error for non-Swagger specification', async () => {
352 |       const openApiSpec = {
353 |         openapi: '3.0.0',
354 |         info: { title: 'Test', version: '1.0.0' }
355 |       };
356 |
357 |       await expect(parser.convertSwaggerToOpenAPI(openApiSpec))
358 |         .rejects.toThrow('Input is not a valid Swagger 2.0 specification');
359 |     });
360 |   });
361 |
```

```
362 |   describe('cache management', () => {
363 |     it('should cache parsed specifications', async () => {
364 |       const spec = {
365 |         openapi: '3.0.0',
366 |         info: { title: 'Test', version: '1.0.0' },
367 |         paths: {}
368 |       };
369 |
370 |       // Parse twice
371 |       await parser.parseSpecification(spec);
372 |       await parser.parseSpecification(spec);
373 |
374 |       const stats = parser.getStats();
375 |       expect(stats.cachedSpecs).toBe(0); // Object specs are not cached
376 |     });
377 |
378 |     it('should clear cache', () => {
379 |       parser.clearCache();
380 |       const stats = parser.getStats();
381 |       expect(stats.cachedSpecs).toBe(0);
382 |     });
383 |   });
384 | });
385 |
```

/tests/unit/plugin-system.test.ts:

```
  1 | /**
  2 |  * Plugin System Unit Tests
  3 |  */
  4 |
  5 | import { describe, it, expect, beforeEach, afterEach, jest } from '@jest/globals';
  6 | import * as fs from 'fs/promises';
  7 | import * as path from 'path';
  8 | import {
  9 |   PluginManager,
 10 |   PluginDiscovery,
 11 |   PluginLoader,
 12 |   BasePlugin,
 13 |   SimplePlugin,
 14 |   PluginSystemFactory,
 15 |   PluginUtils,
 16 |   DEFAULT_PLUGIN_CONFIG
 17 | } from '../../src/core/plugin-system';
 18 | import type {
```

```
19 |   PluginMetadata,
20 |   PluginLoaderConfig,
21 |   PluginManifest
22 | } from '../../src/core/plugin-system/types';
23 | import { Logger } from '../../src/utils/logger';
24 |
25 | // Mock file system operations
26 | jest.mock('fs/promises');
27 | const mockFs = fs as jest.Mocked<typeof fs>;
28 |
29 | describe('Plugin System', () => {
30 |   let testConfig: PluginLoaderConfig;
31 |   let logger: Logger;
32 |
33 |   beforeEach(() => {
34 |     testConfig = {
35 |       ...DEFAULT_PLUGIN_CONFIG,
36 |       pluginsDir: './test-plugins',
37 |       dataDir: './test-data'
38 |     };
39 |     logger = new Logger({ name: 'test', level: 'error' });
40 |     jest.clearAllMocks();
41 |   });
42 |
43 |   afterEach(() => {
44 |     jest.resetAllMocks();
45 |   });
46 |
47 |   describe('PluginUtils', () => {
48 |     describe('validateMetadata', () => {
49 |       it('should return no errors for valid metadata', () => {
50 |         const metadata: PluginMetadata = {
51 |           id: 'test-plugin',
52 |           name: 'Test Plugin',
53 |           version: '1.0.0',
54 |           description: 'A test plugin',
55 |           author: 'Test Author'
56 |         };
57 |
58 |         const errors = PluginUtils.validateMetadata(metadata);
59 |         expect(errors).toHaveLength(0);
60 |       });
61 |
62 |       it('should return errors for missing required fields', () => {
63 |         const metadata = {};
64 |         const errors = PluginUtils.validateMetadata(metadata);
65 |
```

```
66 |         expect(errors).toContain('Plugin ID is required');
67 |         expect(errors).toContain('Plugin name is required');
68 |         expect(errors).toContain('Plugin version is required');
69 |         expect(errors).toContain('Plugin description is required');
70 |         expect(errors).toContain('Plugin author is required');
71 |       });
72 |     });
73 |
74 |     describe('compareVersions', () => {
75 |       it('should compare versions correctly', () => {
76 |         expect(PluginUtils.compareVersions('1.0.0', '1.0.0')).toBe(0);
77 |         expect(PluginUtils.compareVersions('1.0.1', '1.0.0')).toBe(1);
78 |         expect(PluginUtils.compareVersions('1.0.0', '1.0.1')).toBe(-1);
79 |         expect(PluginUtils.compareVersions('2.0.0', '1.9.9')).toBe(1);
80 |       });
81 |     });
82 |
83 |     describe('satisfiesVersion', () => {
84 |       it('should check version ranges correctly', () => {
85 |         expect(PluginUtils.satisfiesVersion('1.0.0', '1.0.0')).toBe(true);
86 |         expect(PluginUtils.satisfiesVersion('1.0.1', '^1.0.0')).toBe(true);
87 |         expect(PluginUtils.satisfiesVersion('1.1.0', '~1.0.0')).toBe(true);
88 |         expect(PluginUtils.satisfiesVersion('0.9.0', '^1.0.0')).toBe(false);
89 |       });
90 |     });
91 |
92 |     describe('generateManifestTemplate', () => {
93 |       it('should generate valid manifest template', () => {
94 |         const manifest = PluginUtils.generateManifestTemplate('test-id', 'Test
Plugin');
95 |
96 |         expect(manifest.id).toBe('test-id');
97 |         expect(manifest.name).toBe('Test Plugin');
98 |         expect(manifest.version).toBe('1.0.0');
99 |         expect(manifest.description).toBe('Test Plugin plugin');
100 |         expect(manifest.main).toBe('index.js');
101 |       });
102 |     });
103 |   });
104 |
105 |   describe('BasePlugin', () => {
106 |     class TestPlugin extends BasePlugin {
107 |       protected async onInitialize(): Promise<void> {
108 |         // Test implementation
109 |       }
110 |
111 |       protected async onStart(): Promise<void> {
```

```
112 |         // Test implementation
113 |       }
114 |
115 |       protected async onStop(): Promise<void> {
116 |         // Test implementation
117 |       }
118 |
119 |       protected async onCleanup(): Promise<void> {
120 |         // Test implementation
121 |       }
122 |     }
123 |
124 |     let plugin: TestPlugin;
125 |     let metadata: PluginMetadata;
126 |
127 |     beforeEach(() => {
128 |       metadata = {
129 |         id: 'test-plugin',
130 |         name: 'Test Plugin',
131 |         version: '1.0.0',
132 |         description: 'A test plugin',
133 |         author: 'Test Author'
134 |       };
135 |       plugin = new TestPlugin(metadata);
136 |     });
137 |
138 |     it('should initialize with correct metadata', () => {
139 |       expect(plugin.metadata).toEqual(metadata);
140 |       expect(plugin.state).toBe('unloaded');
141 |     });
142 |
143 |     it('should transition states correctly during lifecycle', async () => {
144 |       const mockContext = {
145 |         metadata,
146 |         config: { enabled: true },
147 |         logger,
148 |         dataDir: './test-data',
149 |         api: {
150 |           registerHook: jest.fn(),
151 |           unregisterHook: jest.fn(),
152 |           emitHook: jest.fn(),
153 |           getPlugin: jest.fn(),
154 |           getPlugins: jest.fn(),
155 |           getHostConfig: jest.fn(),
156 |           registerService: jest.fn(),
157 |           getService: jest.fn(),
158 |           log: jest.fn()
```

```
159 |         },
160 |         events: { on: jest.fn(), emit: jest.fn() }
161 |       } as any;
162 |
163 |       // Initialize
164 |       await plugin.initialize(mockContext);
165 |       expect(plugin.state).toBe('initialized');
166 |
167 |       // Start
168 |       await plugin.start();
169 |       expect(plugin.state).toBe('started');
170 |
171 |       // Stop
172 |       await plugin.stop();
173 |       expect(plugin.state).toBe('stopped');
174 |
175 |       // Cleanup
176 |       await plugin.cleanup();
177 |       expect(plugin.state).toBe('unloaded');
178 |     });
179 |
180 |     it('should provide health status', () => {
181 |       const health = plugin.getHealth();
182 |
183 |       expect(health.status).toBe('unknown');
184 |       expect(health.timestamp).toBeInstanceOf(Date);
185 |       expect(health.details).toBeDefined();
186 |     });
187 |   });
188 |
189 |   describe('SimplePlugin', () => {
190 |     it('should create plugin with handlers', async () => {
191 |       const initHandler = jest.fn();
192 |       const startHandler = jest.fn();
193 |       const stopHandler = jest.fn();
194 |       const cleanupHandler = jest.fn();
195 |
196 |       const metadata: PluginMetadata = {
197 |         id: 'simple-plugin',
198 |         name: 'Simple Plugin',
199 |         version: '1.0.0',
200 |         description: 'A simple plugin',
201 |         author: 'Test Author'
202 |       };
203 |
204 |       const plugin = new SimplePlugin(metadata, {
205 |         onInitialize: initHandler,
```

```
206 |         onStart: startHandler,
207 |         onStop: stopHandler,
208 |         onCleanup: cleanupHandler
209 |       });
210 |
211 |       const mockContext = {
212 |         metadata,
213 |         config: { enabled: true },
214 |         logger,
215 |         dataDir: './test-data',
216 |         api: {
217 |           registerHook: jest.fn(),
218 |           unregisterHook: jest.fn(),
219 |           emitHook: jest.fn(),
220 |           getPlugin: jest.fn(),
221 |           getPlugins: jest.fn(),
222 |           getHostConfig: jest.fn(),
223 |           registerService: jest.fn(),
224 |           getService: jest.fn(),
225 |           log: jest.fn()
226 |         },
227 |         events: { on: jest.fn(), emit: jest.fn() }
228 |       } as any;
229 |
230 |       await plugin.initialize(mockContext);
231 |       expect(initHandler).toHaveBeenCalled();
232 |
233 |       await plugin.start();
234 |       expect(startHandler).toHaveBeenCalled();
235 |
236 |       await plugin.stop();
237 |       expect(stopHandler).toHaveBeenCalled();
238 |
239 |       await plugin.cleanup();
240 |       expect(cleanupHandler).toHaveBeenCalled();
241 |     });
242 |   });
243 |
244 |   describe('PluginDiscovery', () => {
245 |     let discovery: PluginDiscovery;
246 |
247 |     beforeEach(() => {
248 |       discovery = new PluginDiscovery(testConfig, logger);
249 |     });
250 |
251 |     it('should validate manifest correctly', async () => {
252 |       const validManifest: PluginManifest = {
```

```
253 |         id: 'test-plugin',
254 |         name: 'Test Plugin',
255 |         version: '1.0.0',
256 |         description: 'A test plugin',
257 |         author: 'Test Author'
258 |       };
259 |
260 |       const errors = await discovery.validateManifest(validManifest);
261 |       expect(errors).toHaveLength(0);
262 |     });
263 |
264 |     it('should detect invalid manifest', async () => {
265 |       const invalidManifest = {
266 |         id: '',
267 |         name: '',
268 |         version: 'invalid',
269 |         description: '',
270 |         author: ''
271 |       } as PluginManifest;
272 |
273 |       const errors = await discovery.validateManifest(invalidManifest);
274 |       expect(errors.length).toBeGreaterThan(0);
275 |     });
276 |
277 |     it('should discover plugins in directory', async () => {
278 |       // Mock file system
279 |       mockFs.readdir.mockResolvedValue([
280 |         { name: 'plugin1', isDirectory: () => true },
281 |         { name: 'plugin2', isDirectory: () => true },
282 |         { name: 'file.txt', isDirectory: () => false }
283 |       ] as any);
284 |
285 |       mockFs.access.mockImplementation(async (path: any) => {
286 |         if (path.includes('plugin.json')) {
287 |           return Promise.resolve();
288 |         }
289 |         throw new Error('File not found');
290 |       });
291 |
292 |       mockFs.readFile.mockResolvedValue(JSON.stringify({
293 |         id: 'test-plugin',
294 |         name: 'Test Plugin',
295 |         version: '1.0.0',
296 |         description: 'A test plugin',
297 |         author: 'Test Author',
298 |         main: 'index.js'
299 |       }));
```

```
300 |
301 |       const results = await discovery.discoverPlugins();
302 |       expect(results).toHaveLength(2);
303 |     });
304 |   });
305 |
306 |   describe('PluginSystemFactory', () => {
307 |     it('should create development plugin system', () => {
308 |       const manager = PluginSystemFactory.createDevelopment('./dev-plugins',
logger);
309 |       expect(manager).toBeInstanceOf(PluginManager);
310 |     });
311 |
312 |     it('should create production plugin system', () => {
313 |       const manager = PluginSystemFactory.createProduction('./prod-plugins',
logger);
314 |       expect(manager).toBeInstanceOf(PluginManager);
315 |     });
316 |
317 |     it('should create testing plugin system', () => {
318 |         const manager = PluginSystemFactory.createTesting('./test-plugins',
logger);
319 |       expect(manager).toBeInstanceOf(PluginManager);
320 |     });
321 |   });
322 |
323 |   describe('PluginManager', () => {
324 |     let manager: PluginManager;
325 |
326 |     beforeEach(() => {
327 |       manager = new PluginManager(testConfig, logger);
328 |     });
329 |
330 |     it('should initialize successfully', async () => {
331 |       // Mock discovery
332 |       mockFs.readdir.mockResolvedValue([]);
333 |       mockFs.mkdir.mockResolvedValue(undefined);
334 |       mockFs.access.mockRejectedValue(new Error('Directory not found'));
335 |
336 |       await manager.initialize();
337 |       expect(manager).toBeDefined();
338 |     });
339 |
340 |     it('should get plugin statistics', () => {
341 |       const stats = manager.getStats();
342 |
343 |       expect(stats.totalDiscovered).toBe(0);
```

```
344 |       expect(stats.totalLoaded).toBe(0);
345 |       expect(stats.totalStarted).toBe(0);
346 |       expect(stats.stateBreakdown).toBeDefined();
347 |     });
348 |
349 |     it('should handle plugin not found error', async () => {
350 |       |                                                                    await
expect(manager.loadPlugin('non-existent')).rejects.toThrow('Plugin non-existent not
found');
351 |     });
352 |   });
353 |
354 |   describe('Integration', () => {
355 |     it('should create and use plugin system end-to-end', async () => {
356 |       // Mock file system for plugin discovery
357 |       mockFs.readdir.mockResolvedValue([
358 |         { name: 'test-plugin', isDirectory: () => true }
359 |       ] as any);
360 |
361 |       mockFs.access.mockResolvedValue(undefined);
362 |       mockFs.mkdir.mockResolvedValue(undefined);
363 |
364 |       const manifest = {
365 |         id: 'test-plugin',
366 |         name: 'Test Plugin',
367 |         version: '1.0.0',
368 |         description: 'A test plugin',
369 |         author: 'Test Author',
370 |         main: 'index.js'
371 |       };
372 |
373 |       mockFs.readFile.mockResolvedValue(JSON.stringify(manifest));
374 |
375 |         const manager = PluginSystemFactory.createTesting('./test-plugins',
logger);
376 |       await manager.initialize();
377 |
378 |       const stats = manager.getStats();
379 |       expect(stats.totalDiscovered).toBeGreaterThanOrEqual(0);
380 |     });
381 |   });
382 | });
383 |
```

/tests/unit/security.test.ts:

```
/**
 * Security Module Unit Tests
 *
 * Comprehensive test suite for the Security Module
 */

import { describe, test, expect, beforeEach, afterEach, jest } from '@jest/globals';
import {SecurityManager, SecurityConfig} from '../../src/core/security/index';
import { Logger } from '../../src/utils/logger/index';

// Mock logger
const mockLogger = {
  info: jest.fn(),
  debug: jest.fn(),
  warn: jest.fn(),
  error: jest.fn()
} as any as Logger;

describe('SecurityManager', () => {
  let securityManager: SecurityManager;
  let config: SecurityConfig;

  beforeEach(async () => {
    // Reset all mocks
    jest.clearAllMocks();

    // Create test configuration
    config = {
      authentication: {
        enabled: true,
        tokenExpiry: 3600,
        maxLoginAttempts: 3,
        lockoutDuration: 300,
        requireTwoFactor: false
      },
      authorization: {
        enabled: true,
        defaultRole: 'user',
        roleHierarchy: {
          'admin': ['user', 'viewer'],
          'user': ['viewer'],
          'viewer': []
        }
      },
      encryption: {
```

```
46 |         algorithm: 'aes-256-gcm',
47 |         keyLength: 256,
48 |         ivLength: 16,
49 |         saltLength: 32,
50 |         iterations: 10000 // Reduced for testing
51 |       },
52 |       rateLimit: {
53 |         enabled: true,
54 |         windowMs: 60000,
55 |         maxRequests: 10,
56 |         skipSuccessfulRequests: false
57 |       },
58 |       audit: {
59 |         enabled: true,
60 |         logLevel: 'standard',
61 |         retentionDays: 7
62 |       }
63 |     };
64 |
65 |     securityManager = new SecurityManager(config, mockLogger);
66 |     await securityManager.initialize();
67 |   });
68 |
69 |   afterEach(async () => {
70 |     if (securityManager) {
71 |       await securityManager.shutdown();
72 |     }
73 |   });
74 |
75 |   describe('Initialization', () => {
76 |     test('should initialize successfully', async () => {
77 |         expect(mockLogger.info).toHaveBeenCalledWith('Initializing Security
Manager...');
78 |             expect(mockLogger.info).toHaveBeenCalledWith('Security  Manager
initialized successfully');
79 |     });
80 |
81 |     test('should create default admin user', async () => {
82 |       const stats = securityManager.getSecurityStats();
83 |       expect(stats.activeUsers).toBe(1);
84 |       expect(mockLogger.warn).toHaveBeenCalledWith(
85 |         expect.stringContaining('Default admin user created')
86 |       );
87 |     });
88 |
89 |     test('should throw error when not initialized', async () => {
90 |       const uninitializedManager = new SecurityManager(config, mockLogger);
```

```
91 |
92 |       await expect(
93 |         uninitializedManager.authenticate('admin', 'password')
94 |       ).rejects.toThrow('Security Manager not initialized');
95 |     });
96 |   });
97 |
98 |   describe('Authentication', () => {
99 |     test('should authenticate valid admin user', async () => {
100 |       const result = await securityManager.authenticate('admin', 'admin123');
101 |
102 |       expect(result.success).toBe(true);
103 |       expect(result.user).toBeDefined();
104 |       expect(result.session).toBeDefined();
105 |       expect(result.token).toBeDefined();
106 |       expect(result.user?.username).toBe('admin');
107 |     });
108 |
109 |     test('should reject invalid credentials', async () => {
110 |       const result = await securityManager.authenticate('admin',
'wrongpassword');
111 |
112 |       expect(result.success).toBe(false);
113 |       expect(result.error).toBe('Invalid credentials');
114 |       expect(result.user).toBeUndefined();
115 |       expect(result.session).toBeUndefined();
116 |     });
117 |
118 |     test('should reject non-existent user', async () => {
119 |       const result = await securityManager.authenticate('nonexistent',
'password');
120 |
121 |       expect(result.success).toBe(false);
122 |       expect(result.error).toBe('Invalid credentials');
123 |     });
124 |
125 |     test('should lock account after max failed attempts', async () => {
126 |       // Attempt to login with wrong password multiple times
127 |       for (let i = 0; i < config.authentication.maxLoginAttempts; i++) {
128 |         await securityManager.authenticate('admin', 'wrongpassword');
129 |       }
130 |
131 |       // Next attempt should indicate account is locked
132 |       const result = await securityManager.authenticate('admin',
'wrongpassword');
133 |       expect(result.success).toBe(false);
134 |       expect(result.error).toBe('Account locked');
```

```
135 |     });
136 |
137 |     test('should handle rate limiting', async () => {
138 |       const ipAddress = '192.168.1.1';
139 |
140 |       // Make requests up to the limit
141 |       for (let i = 0; i < config.rateLimit.maxRequests; i++) {
142 |         const result = await securityManager.authenticate('admin', 'admin123',
ipAddress);
143 |         if (i === 0) {
144 |           expect(result.success).toBe(true);
145 |         }
146 |       }
147 |
148 |       // Next request should be rate limited
149 |       const result = await securityManager.authenticate('admin', 'admin123',
ipAddress);
150 |       expect(result.success).toBe(false);
151 |       expect(result.error).toBe('Rate limit exceeded');
152 |     });
153 |   });
154 |
155 |   describe('Session Management', () => {
156 |     test('should validate valid session token', async () => {
157 |       const authResult = await securityManager.authenticate('admin',
'admin123');
158 |       expect(authResult.success).toBe(true);
159 |
160 |       const session = await
securityManager.validateSession(authResult.token!);
161 |       expect(session).toBeDefined();
162 |       expect(session?.userId).toBe(authResult.user?.id);
163 |     });
164 |
165 |     test('should reject invalid session token', async () => {
166 |       const session = await securityManager.validateSession('invalid-token');
167 |       expect(session).toBeNull();
168 |     });
169 |
170 |     test('should revoke session', async () => {
171 |       const authResult = await securityManager.authenticate('admin',
'admin123');
172 |       expect(authResult.success).toBe(true);
173 |
174 |       const revoked = await
securityManager.revokeSession(authResult.session!.id);
175 |       expect(revoked).toBe(true);
```

```
176 |
177     |                                       const     session     =     await
securityManager.validateSession(authResult.token!);
178 |       expect(session).toBeNull();
179 |     });
180 |
181 |     test('should revoke all user sessions', async () => {
182 |       // Create multiple sessions
183 |       const auth1 = await securityManager.authenticate('admin', 'admin123');
184 |       const auth2 = await securityManager.authenticate('admin', 'admin123');
185 |
186 |       expect(auth1.success).toBe(true);
187 |       expect(auth2.success).toBe(true);
188 |
189     |                                   const     revokedCount     =     await
securityManager.revokeUserSessions(auth1.user!.id);
190 |       expect(revokedCount).toBe(2);
191 |
192 |       // Both sessions should be invalid
193 |       const session1 = await securityManager.validateSession(auth1.token!);
194 |       const session2 = await securityManager.validateSession(auth2.token!);
195 |       expect(session1).toBeNull();
196 |       expect(session2).toBeNull();
197 |     });
198 |   });
199 |
200 |   describe('Authorization', () => {
201 |     test('should authorize admin user for any action', async () => {
202 |           const  authResult  =  await  securityManager.authenticate('admin',
'admin123');
203 |       expect(authResult.success).toBe(true);
204 |
205 |       const authzResult = await securityManager.authorize(
206 |         authResult.user!.id,
207 |         'validation',
208 |         'execute'
209 |       );
210 |
211 |       expect(authzResult.allowed).toBe(true);
212 |     });
213 |
214 |     test('should deny authorization for non-existent user', async () => {
215 |       const authzResult = await securityManager.authorize(
216 |         'non-existent-user',
217 |         'validation',
218 |         'execute'
219 |       );
```

```
220 |
221 |       expect(authzResult.allowed).toBe(false);
222 |       expect(authzResult.reason).toBe('User not found');
223 |     });
224 |
225 |     test('should respect authorization when disabled', async () => {
226 |       // Create manager with authorization disabled
227 |       const disabledConfig = { ...config };
228 |       disabledConfig.authorization.enabled = false;
229 |
230 |       const disabledManager = new SecurityManager(disabledConfig, mockLogger);
231 |       await disabledManager.initialize();
232 |
233 |       const authzResult = await disabledManager.authorize(
234 |         'any-user',
235 |         'any-resource',
236 |         'any-action'
237 |       );
238 |
239 |       expect(authzResult.allowed).toBe(true);
240 |
241 |       await disabledManager.shutdown();
242 |     });
243 |   });
244 |
245 |   describe('Encryption', () => {
246 |     test('should encrypt and decrypt data successfully', () => {
247 |       const originalData = 'sensitive information';
248 |       const password = 'encryption-password';
249 |
250 |       const encrypted = securityManager.encrypt(originalData, password);
251 |       expect(encrypted.encrypted).toBeDefined();
252 |       expect(encrypted.iv).toBeDefined();
253 |       expect(encrypted.salt).toBeDefined();
254 |
255 |       const decrypted = securityManager.decrypt(
256 |         encrypted.encrypted,
257 |         encrypted.iv,
258 |         encrypted.salt,
259 |         password,
260 |         encrypted.authTag
261 |       );
262 |
263 |       expect(decrypted).toBe(originalData);
264 |     });
265 |
266 |     test('should fail to decrypt with wrong password', () => {
```

```
267 |       const originalData = 'sensitive information';
268 |       const correctPassword = 'correct-password';
269 |       const wrongPassword = 'wrong-password';
270 |
271 |       const encrypted = securityManager.encrypt(originalData, correctPassword);
272 |
273 |       expect(() => {
274 |         securityManager.decrypt(
275 |           encrypted.encrypted,
276 |           encrypted.iv,
277 |           encrypted.salt,
278 |           wrongPassword
279 |         );
280 |       }).toThrow();
281 |     });
282 |
283 |     test('should encrypt without password using default key', () => {
284 |       const originalData = 'test data';
285 |
286 |       const encrypted = securityManager.encrypt(originalData);
287 |       expect(encrypted.encrypted).toBeDefined();
288 |
289 |       const decrypted = securityManager.decrypt(
290 |         encrypted.encrypted,
291 |         encrypted.iv,
292 |         encrypted.salt,
293 |         undefined,
294 |         encrypted.authTag
295 |       );
296 |
297 |       expect(decrypted).toBe(originalData);
298 |     });
299 |   });
300 |
301 |   describe('Input Validation and Sanitization', () => {
302 |     test('should sanitize malicious input', () => {
303 |       const maliciousInput = '<script>alert("xss")</script>';
304 |       const sanitized = securityManager.sanitizeInput(maliciousInput);
305 |
306 |       expect(sanitized).not.toContain('<');
307 |       expect(sanitized).not.toContain('>');
308 |       expect(sanitized).toBe('scriptalert("xss")/script');
309 |     });
310 |
311 |     test('should validate email format', () => {
312 |       expect(securityManager.validateInput('test@example.com',
```

```
'email')).toBe(true);
313   |                       expect(securityManager.validateInput('invalid-email',
'email')).toBe(false);
314 |       expect(securityManager.validateInput('test@', 'email')).toBe(false);
315 |     });
316 |
317 |     test('should validate username format', () => {
318   |                         expect(securityManager.validateInput('validuser123',
'username')).toBe(true);
319   |                           expect(securityManager.validateInput('user_name',
'username')).toBe(true);
320 |       expect(securityManager.validateInput('ab', 'username')).toBe(false); //
too short
321   |                           expect(securityManager.validateInput('user@name',
'username')).toBe(false); // invalid chars
322 |     });
323 |
324 |     test('should validate password strength', () => {
325   |                       expect(securityManager.validateInput('StrongPass123!',
'password')).toBe(true);
326   |                             expect(securityManager.validateInput('weakpass',
'password')).toBe(false);
327   |                           expect(securityManager.validateInput('NoNumbers!',
'password')).toBe(false);
328 |     });
329 |   });
330 |
331 |   describe('Security Events and Audit', () => {
332 |     test('should log security events', async () => {
333 |       await securityManager.authenticate('admin', 'admin123');
334 |
335 |       const events = securityManager.getSecurityEvents();
336 |       expect(events.length).toBeGreaterThan(0);
337 |
338 |       const authEvent = events.find(e => e.type === 'authentication');
339 |       expect(authEvent).toBeDefined();
340 |       expect(authEvent?.message).toContain('Authentication successful');
341 |     });
342 |
343 |     test('should filter security events', async () => {
344 |       await securityManager.authenticate('admin', 'admin123');
345 |       await securityManager.authenticate('admin', 'wrongpassword');
346 |
347   |             const  authEvents  =  securityManager.getSecurityEvents({  type:
'authentication' });
348 |       expect(authEvents.length).toBeGreaterThan(0);
349 |       expect(authEvents.every(e => e.type === 'authentication')).toBe(true);
```

```
350 |     });
351 |
352 |     test('should limit security events', async () => {
353 |       await securityManager.authenticate('admin', 'admin123');
354 |       await securityManager.authenticate('admin', 'wrongpassword');
355 |
356 |       const limitedEvents = securityManager.getSecurityEvents({ limit: 1 });
357 |       expect(limitedEvents.length).toBe(1);
358 |     });
359 |
360 |     test('should provide security statistics', () => {
361 |       const stats = securityManager.getSecurityStats();
362 |
363 |       expect(stats).toHaveProperty('activeUsers');
364 |       expect(stats).toHaveProperty('activeSessions');
365 |       expect(stats).toHaveProperty('securityEvents');
366 |       expect(stats).toHaveProperty('rateLimitEntries');
367 |       expect(typeof stats.activeUsers).toBe('number');
368 |     });
369 |   });
370 | });
371 |
```

/tests/unit/simple-mcp-server.test.ts:

```
  1 | /**
  2 |  * Simple MCP Server Tests
  3 |  *
  4 |  * Test suite for the simplified MCP Protocol Server implementation
  5 |  */
  6 |
  7 | import { describe, test, expect, beforeEach, afterEach } from '@jest/globals';
  8 | import request from 'supertest';
  9 | import WebSocket from 'ws';
 10 |   |   import   {   SimpleMCPServer,   SimpleMCPServerConfig   }   from
'../../src/core/mcp-server/simple-server';
 11 |
 12 | describe('Simple MCP Server', () => {
 13 |   let server: SimpleMCPServer;
 14 |   let config: SimpleMCPServerConfig;
 15 |
 16 |   beforeEach(() => {
 17 |     config = {
 18 |       name: 'test-simple-mcp-server',
 19 |       version: '1.0.0',
```

```
20 |       port: 3002,
21 |       enableCors: true,
22 |       enableWebSocket: true
23 |     };
24 |     server = new SimpleMCPServer(config);
25 |   });
26 |
27 |   afterEach(async () => {
28 |     if (server) {
29 |       await server.stop();
30 |     }
31 |   });
32 |
33 |   describe('Server Lifecycle', () => {
34 |     test('should start and stop server successfully', async () => {
35 |       await server.start();
36 |       expect(server.getStatus().isRunning).toBe(true);
37 |
38 |       await server.stop();
39 |       expect(server.getStatus().isRunning).toBe(false);
40 |     });
41 |
42 |     test('should not start server twice', async () => {
43 |       await server.start();
44 |       await expect(server.start()).rejects.toThrow('Server is already
running');
45 |     });
46 |   });
47 |
48 |   describe('HTTP Endpoints', () => {
49 |     beforeEach(async () => {
50 |       await server.start();
51 |     });
52 |
53 |     test('should respond to health check', async () => {
54 |       const response = await request(`http://localhost:${config.port}`)
55 |         .get('/health')
56 |         .expect(200);
57 |
58 |       expect(response.body).toMatchObject({
59 |         status: 'healthy',
60 |         server: config.name,
61 |         version: config.version
62 |       });
63 |     });
64 |
65 |     test('should respond to server info', async () => {
```

```
66 |       const response = await request(`http://localhost:${config.port}`)
67 |         .get('/info')
68 |         .expect(200);
69 |
70 |       expect(response.body).toMatchObject({
71 |         name: config.name,
72 |         version: config.version,
73 |         protocolVersion: '2024-11-05'
74 |       });
75 |     });
76 |
77 |     test('should handle initialize request', async () => {
78 |       const initRequest = {
79 |         jsonrpc: '2.0',
80 |         id: 1,
81 |         method: 'initialize',
82 |         params: {
83 |           protocolVersion: '2024-11-05',
84 |           capabilities: {
85 |             tools: {}
86 |           },
87 |           clientInfo: {
88 |             name: 'test-client',
89 |             version: '1.0.0'
90 |           }
91 |         }
92 |       };
93 |
94 |       const response = await request(`http://localhost:${config.port}`)
95 |         .post('/mcp')
96 |         .send(initRequest)
97 |         .expect(200);
98 |
99 |       expect(response.body).toMatchObject({
100 |         jsonrpc: '2.0',
101 |         id: 1,
102 |         result: {
103 |           protocolVersion: '2024-11-05',
104 |           serverInfo: {
105 |             name: config.name,
106 |             version: config.version
107 |           }
108 |         }
109 |       });
110 |
111 |       expect(response.headers['mcp-session-id']).toBeDefined();
112 |     });
```

```
113 |
114 |     test('should reject request without session ID', async () => {
115 |       const request_data = {
116 |         jsonrpc: '2.0',
117 |         id: 1,
118 |         method: 'tools/list'
119 |       };
120 |
121 |       const response = await request(`http://localhost:${config.port}`)
122 |         .post('/mcp')
123 |         .send(request_data)
124 |         .expect(400);
125 |
126 |           expect(response.body.error.message).toContain('No valid session ID
provided');
127 |     });
128 |
129 |     test('should handle tools/list request', async () => {
130 |       // First initialize
131 |       const initRequest = {
132 |         jsonrpc: '2.0',
133 |         id: 1,
134 |         method: 'initialize',
135 |         params: {}
136 |       };
137 |
138 |       const initResponse = await request(`http://localhost:${config.port}`)
139 |         .post('/mcp')
140 |         .send(initRequest);
141 |
142 |       const sessionId = initResponse.headers['mcp-session-id'];
143 |
144 |       // Then list tools
145 |       const toolsRequest = {
146 |         jsonrpc: '2.0',
147 |         id: 2,
148 |         method: 'tools/list'
149 |       };
150 |
151 |       const response = await request(`http://localhost:${config.port}`)
152 |         .post('/mcp')
153 |         .set('mcp-session-id', sessionId)
154 |         .send(toolsRequest)
155 |         .expect(200);
156 |
157 |       expect(response.body).toMatchObject({
158 |         jsonrpc: '2.0',
```

```
159 |         id: 2,
160 |         result: {
161 |           tools: expect.any(Array)
162 |         }
163 |       });
164 |
165 |       expect(response.body.result.tools.length).toBeGreaterThan(0);
166 |       expect(response.body.result.tools[0]).toMatchObject({
167 |         name: 'validate-interface',
168 |         description: expect.any(String),
169 |         inputSchema: expect.any(Object)
170 |       });
171 |     });
172 |
173 |     test('should handle tools/call request', async () => {
174 |       // Initialize session
175 |       const initRequest = {
176 |         jsonrpc: '2.0',
177 |         id: 1,
178 |         method: 'initialize',
179 |         params: {}
180 |       };
181 |
182 |       const initResponse = await request(`http://localhost:${config.port}`)
183 |         .post('/mcp')
184 |         .send(initRequest);
185 |
186 |       const sessionId = initResponse.headers['mcp-session-id'];
187 |
188 |       // Call validation tool
189 |       const callRequest = {
190 |         jsonrpc: '2.0',
191 |         id: 2,
192 |         method: 'tools/call',
193 |         params: {
194 |           name: 'validate-interface',
195 |           arguments: {
196 |             frontendCode: 'const api = { getData: () => Promise<string> }',
197 |             backendCode: 'app.get("/data", (req, res) => res.json("test"))'
198 |           }
199 |         }
200 |       };
201 |
202 |       const response = await request(`http://localhost:${config.port}`)
203 |         .post('/mcp')
204 |         .set('mcp-session-id', sessionId)
205 |         .send(callRequest)
```

```
206 |         .expect(200);
207 |
208 |       expect(response.body).toMatchObject({
209 |         jsonrpc: '2.0',
210 |         id: 2,
211 |         result: {
212 |           content: expect.any(Array)
213 |         }
214 |       });
215 |
216 |       const content = JSON.parse(response.body.result.content[0].text);
217 |       expect(content).toMatchObject({
218 |         isValid: true,
219 |         score: expect.any(Number),
220 |         issues: expect.any(Array),
221 |         suggestions: expect.any(Array)
222 |       });
223 |     });
224 |
225 |     test('should handle unknown method', async () => {
226 |       // Initialize session
227 |       const initRequest = {
228 |         jsonrpc: '2.0',
229 |         id: 1,
230 |         method: 'initialize',
231 |         params: {}
232 |       };
233 |
234 |       const initResponse = await request(`http://localhost:${config.port}`)
235 |         .post('/mcp')
236 |         .send(initRequest);
237 |
238 |       const sessionId = initResponse.headers['mcp-session-id'];
239 |
240 |       // Call unknown method
241 |       const unknownRequest = {
242 |         jsonrpc: '2.0',
243 |         id: 2,
244 |         method: 'unknown/method'
245 |       };
246 |
247 |       const response = await request(`http://localhost:${config.port}`)
248 |         .post('/mcp')
249 |         .set('mcp-session-id', sessionId)
250 |         .send(unknownRequest)
251 |         .expect(200);
252 |
```

```
253 |       expect(response.body).toMatchObject({
254 |         jsonrpc: '2.0',
255 |         id: 2,
256 |         error: {
257 |           code: -32601,
258 |           message: 'Method not found: unknown/method'
259 |         }
260 |       });
261 |     });
262 |
263 |     test('should handle session termination', async () => {
264 |       // Initialize session
265 |       const initRequest = {
266 |         jsonrpc: '2.0',
267 |         id: 1,
268 |         method: 'initialize',
269 |         params: {}
270 |       };
271 |
272 |       const initResponse = await request(`http://localhost:${config.port}`)
273 |         .post('/mcp')
274 |         .send(initRequest);
275 |
276 |       const sessionId = initResponse.headers['mcp-session-id'];
277 |
278 |       // Terminate session
279 |       const response = await request(`http://localhost:${config.port}`)
280 |         .delete('/mcp')
281 |         .set('mcp-session-id', sessionId)
282 |         .expect(200);
283 |
284 |       expect(response.body).toMatchObject({
285 |         jsonrpc: '2.0',
286 |         result: {
287 |           message: 'Session terminated successfully'
288 |         }
289 |       });
290 |     });
291 |   });
292 |
293 |   describe('WebSocket Support', () => {
294 |     beforeEach(async () => {
295 |       await server.start();
296 |     });
297 |
298 |     test('should accept WebSocket connections', (done) => {
299 |       const ws = new WebSocket(`ws://localhost:${config.port}/ws`);
```

```
300 |
301 |       ws.on('open', () => {
302 |         expect(ws.readyState).toBe(WebSocket.OPEN);
303 |         ws.close();
304 |         done();
305 |       });
306 |
307 |       ws.on('error', (error) => {
308 |         done(error);
309 |       });
310 |     });
311 |
312 |     test('should handle WebSocket JSON-RPC messages', (done) => {
313 |       const ws = new WebSocket(`ws://localhost:${config.port}/ws`);
314 |
315 |       ws.on('open', () => {
316 |         const initRequest = {
317 |           jsonrpc: '2.0',
318 |           id: 1,
319 |           method: 'initialize',
320 |           params: {
321 |             clientInfo: {
322 |               name: 'websocket-client',
323 |               version: '1.0.0'
324 |             }
325 |           }
326 |         };
327 |
328 |         ws.send(JSON.stringify(initRequest));
329 |       });
330 |
331 |       ws.on('message', (data) => {
332 |         const response = JSON.parse(data.toString());
333 |
334 |         expect(response).toMatchObject({
335 |           jsonrpc: '2.0',
336 |           id: 1,
337 |           result: {
338 |             protocolVersion: '2024-11-05',
339 |             serverInfo: {
340 |               name: config.name,
341 |               version: config.version
342 |             }
343 |           }
344 |         });
345 |
346 |         ws.close();
```

```
347 |         done();
348 |       });
349 |
350 |       ws.on('error', (error) => {
351 |         done(error);
352 |       });
353 |     });
354 |   });
355 |
356 |   describe('Session Management', () => {
357 |     beforeEach(async () => {
358 |       await server.start();
359 |     });
360 |
361 |     test('should track active sessions', async () => {
362 |       const initialSessions = server.getSessions().length;
363 |
364 |       // Create a session
365 |       const initRequest = {
366 |         jsonrpc: '2.0',
367 |         id: 1,
368 |         method: 'initialize',
369 |         params: {}
370 |       };
371 |
372 |       await request(`http://localhost:${config.port}`)
373 |         .post('/mcp')
374 |         .send(initRequest);
375 |
376 |       expect(server.getSessions().length).toBe(initialSessions + 1);
377 |     });
378 |
379 |     test('should provide server status', () => {
380 |       const status = server.getStatus();
381 |
382 |       expect(status).toMatchObject({
383 |         name: config.name,
384 |         version: config.version,
385 |         isRunning: true,
386 |         port: config.port,
387 |         sessions: expect.any(Number)
388 |       });
389 |     });
390 |   });
391 | });
392 |
```

/tests/unit/trae-adapter.test.ts:

```
  1 | /**
  2 |  * Trae Adapter Unit Tests
  3 |  *
  4 |  * Comprehensive test suite for the Trae Tool Adapter with MCP integration
  5 |  */
  6 |
  7 | import { TraeAdapter, TraeAdapterConfig, TraeContextData } from
'../../src/adapters/trae';
  8 | import { ValidationEngine } from '../../src/core/validation';
  9 | import { ContextEnhancer } from '../../src/core/context';
 10 | import { SpecParser } from '../../src/core/parser';
 11 |
 12 | // Mock dependencies
 13 | jest.mock('../../src/core/validation');
 14 | jest.mock('../../src/core/context');
 15 | jest.mock('../../src/core/parser');
 16 | jest.mock('../../src/utils/logger');
 17 |
 18 | describe('TraeAdapter', () => {
 19 |   let adapter: TraeAdapter;
 20 |   let mockConfig: TraeAdapterConfig;
 21 |   let mockValidationEngine: jest.Mocked<ValidationEngine>;
 22 |   let mockContextEnhancer: jest.Mocked<ContextEnhancer>;
 23 |   let mockSpecParser: jest.Mocked<SpecParser>;
 24 |
 25 |   beforeEach(() => {
 26 |     // Reset all mocks
 27 |     jest.clearAllMocks();
 28 |
 29 |     // Create mock config
 30 |     mockConfig = {
 31 |       enableRealTimeValidation: true,
 32 |       contextInjectionMode: 'auto',
 33 |       validationTriggers: ['save', 'change'],
 34 |       mcpServerPath: '/path/to/trae/server',
 35 |       serverCommand: 'node',
 36 |       serverArgs: ['server.js'],
 37 |       workspaceRoot: '/workspace',
 38 |       capabilities: {
 39 |         tools: true,
 40 |         resources: true,
 41 |         prompts: true,
 42 |         logging: true,
```

```
43 |         sampling: false
44 |       },
45 |       requestTimeout: 30000
46 |     };
47 |
48 |     // Create mock instances
49   |                 mockValidationEngine   =   new   ValidationEngine()   as
jest.Mocked<ValidationEngine>;
50   |                  mockContextEnhancer   =   new   ContextEnhancer()   as
jest.Mocked<ContextEnhancer>;
51 |     mockSpecParser = new SpecParser() as jest.Mocked<SpecParser>;
52 |
53 |     // Setup mock implementations
54 |     mockValidationEngine.initialize = jest.fn().mockResolvedValue(undefined);
55 |     mockValidationEngine.validateInterface = jest.fn().mockResolvedValue({
56 |       isValid: true,
57 |       errors: [],
58 |       warnings: [],
59 |       suggestions: []
60 |     });
61 |
62 |     mockContextEnhancer.initialize = jest.fn().mockResolvedValue(undefined);
63 |     mockContextEnhancer.generateContext = jest.fn().mockResolvedValue({
64 |       id: 'test-context',
65 |       content: 'Generated context'
66 |     });
67 |
68 |     mockSpecParser.parseSpec = jest.fn().mockResolvedValue({
69 |       success: true,
70 |       spec: { openapi: '3.0.0', info: { title: 'Test API' } },
71 |       errors: []
72 |     });
73 |
74 |     // Create adapter with dependency injection
75   |           adapter   =   new   TraeAdapter(mockConfig,   mockValidationEngine,
mockContextEnhancer, mockSpecParser);
76 |   });
77 |
78 |   afterEach(() => {
79 |     if (adapter) {
80 |       adapter.dispose();
81 |     }
82 |   });
83 |
84 |   describe('Initialization', () => {
85 |     it('should initialize successfully with valid config', async () => {
86 |       await expect(adapter.initialize()).resolves.not.toThrow();
```

```
 87 |
 88 |       expect(mockValidationEngine.initialize).toHaveBeenCalled();
 89 |       expect(mockContextEnhancer.initialize).toHaveBeenCalled();
 90 |       expect(adapter.getStatistics().isActive).toBe(true);
 91 |     });
 92 |
 93 |     it('should initialize without MCP server path', async () => {
 94 |       const configWithoutMCP = { ...mockConfig };
 95 |       delete configWithoutMCP.mcpServerPath;
 96 |
 97 |       const adapterWithoutMCP = new TraeAdapter(configWithoutMCP,
mockValidationEngine, mockContextEnhancer, mockSpecParser);
 98 |
 99 |       await expect(adapterWithoutMCP.initialize()).resolves.not.toThrow();
100 |       expect(adapterWithoutMCP.getStatistics().mcpConnected).toBe(false);
101 |
102 |       adapterWithoutMCP.dispose();
103 |     });
104 |
105 |     it('should handle initialization errors gracefully', async () => {
106 |       mockValidationEngine.initialize.mockRejectedValue(new Error('Validation
engine failed'));
107 |
108 |       await expect(adapter.initialize()).rejects.toThrow('Validation engine
failed');
109 |     });
110 |
111 |     it('should emit initialized event on successful initialization', async ()
=> {
112 |       const initSpy = jest.fn();
113 |       adapter.on('initialized', initSpy);
114 |
115 |       await adapter.initialize();
116 |
117 |       expect(initSpy).toHaveBeenCalled();
118 |     });
119 |
120 |     it('should set up MCP capabilities correctly', async () => {
121 |       await adapter.initialize();
122 |
123 |       const tools = adapter.getAvailableTools();
124 |       const resources = adapter.getAvailableResources();
125 |       const prompts = adapter.getAvailablePrompts();
126 |
127 |       expect(tools.length).toBeGreaterThan(0);
128 |       expect(resources.length).toBeGreaterThan(0);
129 |       expect(prompts.length).toBeGreaterThan(0);
```

```
130 |
131 |       // Check specific tools
132 |       expect(tools.find(t => t.name === 'validate_api_spec')).toBeDefined();
133 |       expect(tools.find(t => t.name === 'generate_context')).toBeDefined();
134   |                               expect(tools.find(t    =>    t.name    ===
'analyze_code_patterns')).toBeDefined();
135 |     });
136 |   });
137 |
138 |   describe('MCP Tool Execution', () => {
139 |     beforeEach(async () => {
140 |       await adapter.initialize();
141 |     });
142 |
143 |     it('should execute validate_api_spec tool successfully', async () => {
144 |       const result = await adapter.executeTool('validate_api_spec', {
145 |         spec: 'openapi: 3.0.0\npaths:\n  /test: {}',
146 |         format: 'yaml'
147 |       });
148 |
149 |       expect(result.success).toBe(true);
150 |       expect(result.data.isValid).toBe(true);
151 |       expect(Array.isArray(result.data.errors)).toBe(true);
152 |       expect(Array.isArray(result.data.warnings)).toBe(true);
153 |       expect(Array.isArray(result.data.suggestions)).toBe(true);
154 |     });
155 |
156 |     it('should handle invalid API spec in tool execution', async () => {
157 |       const result = await adapter.executeTool('validate_api_spec', {
158 |         spec: 'invalid spec content',
159 |         format: 'yaml'
160 |       });
161 |
162 |       expect(result.success).toBe(true);
163 |       expect(result.data.isValid).toBe(false);
164 |       expect(result.data.errors.length).toBeGreaterThan(0);
165 |     });
166 |
167 |     it('should execute generate_context tool successfully', async () => {
168 |       const result = await adapter.executeTool('generate_context', {
169 |         language: 'typescript',
170 |         framework: 'express',
171 |         apiType: 'rest'
172 |       });
173 |
174 |       expect(result.success).toBe(true);
175 |       expect(result.data.context).toBeDefined();
```

```
176 |       expect(Array.isArray(result.data.templates)).toBe(true);
177 |       expect(Array.isArray(result.data.recommendations)).toBe(true);
178 |     });
179 |
180 |     it('should handle unknown tool execution', async () => {
181 |       const result = await adapter.executeTool('unknown_tool', {});
182 |
183 |       expect(result.success).toBe(false);
184 |       expect(result.error).toContain('Tool not found');
185 |     });
186 |
187 |     it('should handle tool execution errors', async () => {
188 |       // Create a spy that simulates an error in tool execution
189   |                           const    executeSpy    =    jest.spyOn(adapter,
'executeTool').mockImplementation(async () => {
190 |         return {
191 |           success: false,
192 |           error: 'Tool execution failed'
193 |         };
194 |       });
195 |
196 |       const result = await adapter.executeTool('validate_api_spec', {});
197 |
198 |       expect(result.success).toBe(false);
199 |       expect(result.error).toContain('Tool execution failed');
200 |
201 |       executeSpy.mockRestore();
202 |     });
203 |   });
204 |
205 |   describe('Document Validation', () => {
206 |     let mockContextData: TraeContextData;
207 |
208 |     beforeEach(async () => {
209 |       await adapter.initialize();
210 |
211 |       mockContextData = {
212 |         documentPath: '/test/api.yaml',
213 |         content: 'openapi: 3.0.0\ninfo:\n  title: Test API\npaths:\n  /test:
{}',
214 |         language: 'yaml',
215 |         projectInfo: {
216 |           name: 'test-project',
217 |           version: '1.0.0',
218 |           type: 'api'
219 |         }
220 |       };
```

```
221 |     });
222 |
223 |     it('should validate API specification document successfully', async () =>
{
224 |       const result = await adapter.validateDocument(mockContextData);
225 |
226 |       expect(result.isValid).toBe(true);
227 |       expect(result.timestamp).toBeDefined();
228 |       expect(result.metadata?.toolsUsed).toContain('validate_api_spec');
229 |       expect(Array.isArray(result.errors)).toBe(true);
230 |       expect(Array.isArray(result.warnings)).toBe(true);
231 |       expect(Array.isArray(result.suggestions)).toBe(true);
232 |     });
233 |
234 |     it('should handle non-API specification documents', async () => {
235 |       const nonApiContextData = {
236 |         ...mockContextData,
237 |         content: 'console.log("Hello World");',
238 |         language: 'javascript'
239 |       };
240 |
241 |       const result = await adapter.validateDocument(nonApiContextData);
242 |
243 |       expect(result.timestamp).toBeDefined();
244 |       expect(result.metadata?.toolsUsed).not.toContain('validate_api_spec');
245 |     });
246 |
247 |     it('should use validation cache for repeated requests', async () => {
248 |       const result1 = await adapter.validateDocument(mockContextData);
249 |       const result2 = await adapter.validateDocument(mockContextData);
250 |
251 |       expect(result1).toEqual(result2);
252 |       expect(result1.timestamp).toBe(result2.timestamp);
253 |     });
254 |
255 |     it('should emit validation result event', async () => {
256 |       const validationSpy = jest.fn();
257 |       adapter.on('validationResult', validationSpy);
258 |
259 |       await adapter.validateDocument(mockContextData);
260 |
261 |       expect(validationSpy).toHaveBeenCalled();
262 |       expect(validationSpy.mock.calls[0][0]).toMatchObject({
263 |         isValid: expect.any(Boolean),
264 |         timestamp: expect.any(Number)
265 |       });
266 |     });
```

```
267 |
268 |     it('should handle validation errors gracefully', async () => {
269 |       // Use non-API content to force fallback to parser
270 |       const nonApiContextData = {
271 |         ...mockContextData,
272 |         content: 'invalid content that is not an API spec',
273 |         language: 'text'
274 |       };
275 |
276 |       mockSpecParser.parseSpec.mockResolvedValue({
277 |         success: false,
278 |         errors: ['Failed to parse API specification'],
279 |         spec: null
280 |       });
281 |
282 |       const result = await adapter.validateDocument(nonApiContextData);
283 |
284 |       expect(result.isValid).toBe(false);
285 |       expect(result.errors).toContain('Failed to parse API specification');
286 |     });
287 |
288 |     it('should clear validation cache', () => {
289 |       adapter.clearValidationCache();
290 |
291 |       const stats = adapter.getStatistics();
292 |       expect(stats.cacheSize).toBe(0);
293 |     });
294 |   });
295 |
296 |   describe('Context Injection', () => {
297 |     let mockContextData: TraeContextData;
298 |
299 |     beforeEach(async () => {
300 |       await adapter.initialize();
301 |
302 |       mockContextData = {
303 |         documentPath: '/test/api.yaml',
304 |         content: 'openapi: 3.0.0\ninfo:\n  title: Test API',
305 |         language: 'yaml',
306 |         projectInfo: {
307 |           name: 'test-project',
308 |           version: '1.0.0',
309 |           type: 'api'
310 |         }
311 |       };
312 |     });
313 |
```

```
314 |     it('should inject context successfully', async () => {
315 |       |                                                                  await
expect(adapter.injectContext(mockContextData)).resolves.not.toThrow();
316 |     });
317 |
318 |     it('should emit context injected event', async () => {
319 |       const contextSpy = jest.fn();
320 |       adapter.on('contextInjected', contextSpy);
321 |
322 |       await adapter.injectContext(mockContextData);
323 |
324 |       expect(contextSpy).toHaveBeenCalled();
325 |       expect(contextSpy.mock.calls[0][0]).toMatchObject({
326 |         contextData: mockContextData,
327 |         context: expect.any(Object)
328 |       });
329 |     });
330 |
331 |     it('should fail when adapter is not active', async () => {
332 |       adapter.dispose();
333 |
334 |       await expect(adapter.injectContext(mockContextData))
335 |         .rejects.toThrow('TraeAdapter is not active');
336 |     });
337 |
338 |     it('should handle context injection errors', async () => {
339 |       // Mock an error in context generation
340   |                       const   originalGenerateContext   =   (adapter   as
any).generateContextForDocument;
341   |                       (adapter   as   any).generateContextForDocument   =
jest.fn().mockRejectedValue(new Error('Context generation failed'));
342 |
343 |       await expect(adapter.injectContext(mockContextData))
344 |         .rejects.toThrow('Context generation failed');
345 |     });
346 |   });
347 |
348 |   describe('Real-time Monitoring', () => {
349 |     beforeEach(async () => {
350 |       jest.useFakeTimers();
351 |       await adapter.initialize();
352 |     });
353 |
354 |     afterEach(() => {
355 |       jest.useRealTimers();
356 |     });
357 |
```

```
358 |     it('should emit monitoring events during real-time monitoring', async () =>
{
359 |       const monitoringSpy = jest.fn();
360 |       adapter.on('monitoring', monitoringSpy);
361 |
362 |       // Fast-forward time to trigger monitoring
363 |       jest.advanceTimersByTime(6000);
364 |
365 |       expect(monitoringSpy).toHaveBeenCalled();
366 |       expect(monitoringSpy.mock.calls[0][0]).toMatchObject({
367 |         timestamp: expect.any(Number),
368 |         status: 'active',
369 |         message: expect.any(String)
370 |       });
371 |     });
372 |
373 |     it('should not monitor when adapter is inactive', async () => {
374 |       const monitoringSpy = jest.fn();
375 |       adapter.on('monitoring', monitoringSpy);
376 |
377 |       adapter.dispose();
378 |       jest.advanceTimersByTime(6000);
379 |
380 |       expect(monitoringSpy).not.toHaveBeenCalled();
381 |     });
382 |   });
383 |
384 |   describe('MCP Capabilities', () => {
385 |     beforeEach(async () => {
386 |       await adapter.initialize();
387 |     });
388 |
389 |     it('should return available tools', () => {
390 |       const tools = adapter.getAvailableTools();
391 |
392 |       expect(Array.isArray(tools)).toBe(true);
393 |       expect(tools.length).toBeGreaterThan(0);
394 |       expect(tools[0]).toMatchObject({
395 |         name: expect.any(String),
396 |         description: expect.any(String),
397 |         inputSchema: expect.any(Object)
398 |       });
399 |     });
400 |
401 |     it('should return available resources', () => {
402 |       const resources = adapter.getAvailableResources();
403 |
```

```
404 |       expect(Array.isArray(resources)).toBe(true);
405 |       expect(resources.length).toBeGreaterThan(0);
406 |       expect(resources[0]).toMatchObject({
407 |         uri: expect.any(String),
408 |         name: expect.any(String),
409 |         mimeType: expect.any(String)
410 |       });
411 |     });
412 |
413 |     it('should return available prompts', () => {
414 |       const prompts = adapter.getAvailablePrompts();
415 |
416 |       expect(Array.isArray(prompts)).toBe(true);
417 |       expect(prompts.length).toBeGreaterThan(0);
418 |       expect(prompts[0]).toMatchObject({
419 |         name: expect.any(String),
420 |         description: expect.any(String)
421 |       });
422 |     });
423 |   });
424 |
425 |   describe('Statistics and Configuration', () => {
426 |     it('should return correct statistics before initialization', () => {
427 |       const stats = adapter.getStatistics();
428 |
429 |       expect(stats).toMatchObject({
430 |         isActive: false,
431 |         cacheSize: 0,
432 |         mcpConnected: false,
433 |         availableToolsCount: 0,
434 |         availableResourcesCount: 0,
435 |         availablePromptsCount: 0,
436 |         config: mockConfig
437 |       });
438 |     });
439 |
440 |     it('should return correct statistics after initialization', async () => {
441 |       await adapter.initialize();
442 |
443 |       const stats = adapter.getStatistics();
444 |
445 |       expect(stats).toMatchObject({
446 |         isActive: true,
447 |         cacheSize: 0,
448 |         mcpConnected: true,
449 |         availableToolsCount: expect.any(Number),
450 |         availableResourcesCount: expect.any(Number),
```

```
451 |         availablePromptsCount: expect.any(Number),
452 |         config: mockConfig
453 |       });
454 |
455 |       expect(stats.availableToolsCount).toBeGreaterThan(0);
456 |       expect(stats.availableResourcesCount).toBeGreaterThan(0);
457 |       expect(stats.availablePromptsCount).toBeGreaterThan(0);
458 |     });
459 |   });
460 |
461 |   describe('Disposal', () => {
462 |     it('should dispose resources correctly', async () => {
463 |       await adapter.initialize();
464 |
465 |       adapter.dispose();
466 |
467 |       const stats = adapter.getStatistics();
468 |       expect(stats.isActive).toBe(false);
469 |       expect(stats.cacheSize).toBe(0);
470 |       expect(stats.mcpConnected).toBe(false);
471 |       expect(stats.availableToolsCount).toBe(0);
472 |       expect(stats.availableResourcesCount).toBe(0);
473 |       expect(stats.availablePromptsCount).toBe(0);
474 |     });
475 |
476 |     it('should handle multiple dispose calls safely', async () => {
477 |       await adapter.initialize();
478 |
479 |       adapter.dispose();
480 |       adapter.dispose(); // Should not throw
481 |
482 |       expect(adapter.getStatistics().isActive).toBe(false);
483 |     });
484 |   });
485 |
486 |   describe('Error Handling', () => {
487 |     it('should handle configuration with missing optional fields', () => {
488 |       const minimalConfig: TraeAdapterConfig = {
489 |         enableRealTimeValidation: false,
490 |         contextInjectionMode: 'disabled',
491 |         validationTriggers: []
492 |       };
493 |
494 |       const minimalAdapter = new TraeAdapter(minimalConfig);
495 |
496 |       expect(minimalAdapter.getStatistics().config).toMatchObject({
497 |         ...minimalConfig,
```

```
498 |         requestTimeout: 30000,
499 |         capabilities: {
500 |           tools: true,
501 |           resources: true,
502 |           prompts: true,
503 |           logging: true,
504 |           sampling: false
505 |         }
506 |       });
507 |
508 |       minimalAdapter.dispose();
509 |     });
510 |
511 |     it('should handle validation with malformed content', async () => {
512 |       await adapter.initialize();
513 |
514 |       const malformedContextData: TraeContextData = {
515 |         documentPath: '/test/malformed.yaml',
516 |         content: 'invalid: yaml: content: [unclosed',
517 |         language: 'yaml'
518 |       };
519 |
520 |       const result = await adapter.validateDocument(malformedContextData);
521 |
522 |       expect(result.timestamp).toBeDefined();
523 |       expect(typeof result.isValid).toBe('boolean');
524 |     });
525 |   });
526 | });
527 |
```

/tests/unit/validation.test.ts:

```
  1 | /**
  2 |  * Unit tests for ValidationEngine
  3 |  */
  4 |
  5 | import { ValidationEngine } from '../../src/core/validation/index';
  6 | import { ConfigManager } from '../../src/utils/config/index';
  7 | import { Logger } from '../../src/utils/logger/index';
  8 | import { resolve } from 'path';
  9 |
 10 | describe('ValidationEngine', () => {
 11 |   let validationEngine: ValidationEngine;
 12 |   let configManager: ConfigManager;
```

```
  let logger: Logger;

  beforeEach(() => {
    configManager = new ConfigManager();
    logger = new Logger('ValidationEngine');
    validationEngine = new ValidationEngine(configManager, logger);
  });

  describe('initialization', () => {
    it('should initialize successfully', async () => {
      await expect(validationEngine.initialize()).resolves.not.toThrow();

      const status = validationEngine.getStatus();
      expect(status.initialized).toBe(true);
    });
  });

  describe('interface validation', () => {
    beforeEach(async () => {
      await validationEngine.initialize();
    });

    it('should validate valid API code', async () => {
      const validCode = `
        app.get('/api/users', (req, res) => {
          res.json({ users: [] });
        });

        app.post('/api/users', (req, res) => {
          res.status(201).json({ id: '123', name: 'Test User' });
        });
      `;

      const specPath = resolve('./config/schemas/sample-api.yaml');

      const result = await validationEngine.validateInterface({
        code: validCode,
        specPath,
        type: 'backend'
      });

      expect(result).toBeDefined();
      expect(result.metadata.validationType).toBe('backend');
      expect(Array.isArray(result.errors)).toBe(true);
      expect(Array.isArray(result.warnings)).toBe(true);
      expect(Array.isArray(result.suggestions)).toBe(true);
    });
```

```
60 |
61 |     it('should detect invalid API paths', async () => {
62 |       const invalidCode = `
63 |         app.get('/api/invalid-endpoint', (req, res) => {
64 |           res.json({ data: 'test' });
65 |         });
66 |       `;
67 |
68 |       const specPath = resolve('./config/schemas/sample-api.yaml');
69 |
70 |       const result = await validationEngine.validateInterface({
71 |         code: invalidCode,
72 |         specPath,
73 |         type: 'backend'
74 |       });
75 |
76 |       expect(result.isValid).toBe(false);
77 |       expect(result.errors.length).toBeGreaterThan(0);
78 |
79 |       const pathError = result.errors.find(error => error.code === 'PATH_NOT_IN_SPEC');
80 |       expect(pathError).toBeDefined();
81 |       expect(pathError?.message).toContain('/api/invalid-endpoint');
82 |     });
83 |
84 |     it('should detect invalid HTTP methods', async () => {
85 |       const invalidCode = `
86 |         app.patch('/api/users', (req, res) => {
87 |           res.json({ message: 'Invalid method' });
88 |         });
89 |       `;
90 |
91 |       const specPath = resolve('./config/schemas/sample-api.yaml');
92 |
93 |       const result = await validationEngine.validateInterface({
94 |         code: invalidCode,
95 |         specPath,
96 |         type: 'backend'
97 |       });
98 |
99 |       // Note: PATCH is actually a valid HTTP method, so this test might need adjustment
100 |       // Let's test with a truly invalid method
101 |       const reallyInvalidCode = `
102 |         app.invalid('/api/users', (req, res) => {
103 |           res.json({ message: 'Invalid method' });
104 |         });
```

```
105 |       `;
106 |
107 |       const result2 = await validationEngine.validateInterface({
108 |         code: reallyInvalidCode,
109 |         specPath,
110 |         type: 'backend'
111 |       });
112 |
113   |                     expect(result2.errors.some(error   =>   error.code   ===
'INVALID_HTTP_METHOD')).toBe(true);
114 |     });
115 |
116 |     it('should provide suggestions for similar paths', async () => {
117 |       const typoCode = `
118 |         app.get('/api/user', (req, res) => {
119 |           res.json({ users: [] });
120 |         });
121 |       `;
122 |
123 |       const specPath = resolve('./config/schemas/sample-api.yaml');
124 |
125 |       const result = await validationEngine.validateInterface({
126 |         code: typoCode,
127 |         specPath,
128 |         type: 'backend'
129 |       });
130 |
131 |
132 |
133 |       expect(result.suggestions.length).toBeGreaterThan(0);
134 |       expect(result.suggestions[0].message).toContain('/users');
135 |     });
136 |
137 |     it('should handle TypeScript interface definitions', async () => {
138 |       const tsCode = `
139 |         interface User {
140 |           id: string;
141 |           name: string;
142 |           email: string;
143 |         }
144 |
145 |         interface CreateUserRequest {
146 |           name: string;
147 |           email: string;
148 |         }
149 |       `;
150 |
```

```
151 |       const specPath = resolve('./config/schemas/sample-api.yaml');
152 |
153 |       const result = await validationEngine.validateInterface({
154 |         code: tsCode,
155 |         specPath,
156 |         type: 'frontend'
157 |       });
158 |
159 |       expect(result).toBeDefined();
160 |       expect(result.metadata.validationType).toBe('frontend');
161 |     });
162 |   });
163 |
164 |   describe('file monitoring', () => {
165 |     beforeEach(async () => {
166 |       await validationEngine.initialize();
167 |     });
168 |
169 |     it('should start and stop monitoring', async () => {
170 |       const paths = ['./src/**/*.ts'];
171 |       const specPath = resolve('./config/schemas/sample-api.yaml');
172 |
173 |       await expect(validationEngine.startMonitoring({
174 |         paths,
175 |         specPath
176 |       })).resolves.not.toThrow();
177 |
178 |       const status = validationEngine.getStatus();
179 |       expect(status.monitoring).toBe(true);
180 |
181 |       await expect(validationEngine.stopMonitoring()).resolves.not.toThrow();
182 |     });
183 |   });
184 |
185 |   describe('error handling', () => {
186 |     it('should handle invalid OpenAPI specification', async () => {
187 |       await validationEngine.initialize();
188 |
189 |       const code = `app.get('/api/test', (req, res) => res.json({}));`;
190 |       const invalidSpecPath = './non-existent-spec.yaml';
191 |
192 |       await expect(validationEngine.validateInterface({
193 |         code,
194 |         specPath: invalidSpecPath,
195 |         type: 'backend'
196 |       })).rejects.toThrow();
197 |     });
```

```
198 |
199 |     it('should handle validation before initialization', async () => {
200 |       const code = `app.get('/api/test', (req, res) => res.json({}));`;
201 |       const specPath = resolve('./config/schemas/sample-api.yaml');
202 |
203 |       await expect(validationEngine.validateInterface({
204 |         code,
205 |         specPath,
206 |         type: 'backend'
207 |       })).rejects.toThrow('Validation Engine not initialized');
208 |     });
209 |   });
210 | });
211 |
```

/tests/unit/version-control.test.ts:

```
  1 | /**
  2 |  * Unit tests for Version Control System
  3 |  */
  4 |
  5 | import { describe, it, expect, beforeEach, afterEach, jest } from '@jest/globals';
  6 | import { promises as fs } from 'fs';
  7 | import * as path from 'path';
  8 | import { VersionControlManager } from '../../src/core/version-control';
  9 |       |       import       {       FileSystemProvider       }       from
'../../src/core/version-control/providers/filesystem-provider';
 10 |      |      import      {      VersionControlConfigFactory      }      from
'../../src/core/version-control/config';
 11 | import type { ApiSpecification, VersionEntry, VersionControlConfig } from
'../../src/core/version-control/types';
 12 |
 13 | // Mock logger
 14 | const mockLogger = {
 15 |   info: jest.fn(),
 16 |   warn: jest.fn(),
 17 |   error: jest.fn(),
 18 |   debug: jest.fn()
 19 | };
 20 |
 21 | // Test data
 22 | const testSpec: ApiSpecification = {
 23 |   openapi: '3.0.0',
 24 |   info: {
 25 |     title: 'Test API',
```

```
    version: '1.0.0',
    description: 'Test API specification'
  },
  paths: {
    '/users': {
      get: {
        summary: 'Get users',
        responses: {
          '200': {
            description: 'Success'
          }
        }
      }
    }
  }
};

const testSpecModified: ApiSpecification = {
  ...testSpec,
  paths: {
    '/users': {
      get: {
        summary: 'Get all users',
        responses: {
          '200': {
            description: 'Success'
          }
        }
      },
      post: {
        summary: 'Create user',
        responses: {
          '201': {
            description: 'Created'
          }
        }
      }
    }
  }
};

describe('Version Control System', () => {
  let versionControl: VersionControlManager;
  let config: VersionControlConfig;
  let testDir: string;

  beforeEach(async () => {
```

```
 73 |     // Create temporary test directory
 74 |     testDir = path.join(__dirname, '../../temp/version-control-test');
 75 |     await fs.mkdir(testDir, { recursive: true });
 76 |
 77 |     // Create test configuration
 78 |     config = VersionControlConfigFactory.createTestConfig();
 79 |     config.provider.config = {
 80 |       basePath: testDir,
 81 |       structure: 'hierarchical',
 82 |       naming: {
 83 |         pattern: '{specId}_{versionId}',
 84 |         includeTimestamp: false,
 85 |         includeHash: false
 86 |       }
 87 |     };
 88 |
 89 |     versionControl = new VersionControlManager(config, mockLogger as any);
 90 |     await versionControl.initialize();
 91 |   });
 92 |
 93 |   afterEach(async () => {
 94 |     // Clean up test directory
 95 |     try {
 96 |       await fs.rm(testDir, { recursive: true, force: true });
 97 |     } catch (error) {
 98 |       // Ignore cleanup errors
 99 |     }
100 |   });
101 |
102 |   describe('Initialization', () => {
103 |     it('should initialize successfully', async () => {
104 |           expect(mockLogger.info).toHaveBeenCalledWith('Initializing version
control system...');
105 |         expect(mockLogger.info).toHaveBeenCalledWith('Version control system
initialized successfully');
106 |     });
107 |
108 |     it('should create required directories', async () => {
109 |       const versionsDir = path.join(testDir, 'versions');
110 |       const branchesDir = path.join(testDir, 'branches');
111 |       const metadataDir = path.join(testDir, 'metadata');
112 |
113 |       await expect(fs.access(versionsDir)).resolves.not.toThrow();
114 |       await expect(fs.access(branchesDir)).resolves.not.toThrow();
115 |       await expect(fs.access(metadataDir)).resolves.not.toThrow();
116 |     });
117 |   });
```

```
118 |
119 |   describe('Version Creation', () => {
120 |     it('should create a new version', async () => {
121 |       const version = await versionControl.createVersion('test-spec', testSpec,
{
122 |         author: 'test-user',
123 |         message: 'Initial version'
124 |       });
125 |
126 |       expect(version).toBeDefined();
127 |       expect(version.specId).toBe('test-spec');
128 |       expect(version.version).toBe('1.0.0');
129 |       expect(version.specification).toEqual(testSpec);
130 |       expect(version.metadata.author).toBe('test-user');
131 |       expect(version.metadata.message).toBe('Initial version');
132 |     });
133 |
134 |     it('should increment version numbers', async () => {
135 |       const version1 = await versionControl.createVersion('test-spec',
testSpec);
136 |       const version2 = await versionControl.createVersion('test-spec',
testSpecModified);
137 |
138 |       expect(version1.version).toBe('1.0.0');
139 |       expect(version2.version).toBe('1.0.1');
140 |       expect(version2.parent).toBe(version1.id);
141 |     });
142 |
143 |     it('should calculate specification hash', async () => {
144 |       const version = await versionControl.createVersion('test-spec',
testSpec);
145 |
146 |       expect(version.hash).toBeDefined();
147 |       expect(version.hash).toHaveLength(64); // SHA-256 hash length
148 |     });
149 |
150 |     it('should emit versionCreated event', async () => {
151 |       const eventSpy = jest.fn();
152 |       versionControl.on('versionCreated', eventSpy);
153 |
154 |       const version = await versionControl.createVersion('test-spec',
testSpec);
155 |
156 |       expect(eventSpy).toHaveBeenCalledWith(version);
157 |     });
158 |   });
159 |
```

```
160 |   describe('Version Retrieval', () => {
161 |     let testVersion: VersionEntry;
162 |
163 |     beforeEach(async () => {
164 |       testVersion = await versionControl.createVersion('test-spec', testSpec);
165 |     });
166 |
167 |     it('should retrieve a version by ID', async () => {
168 |       const retrieved = await versionControl.getVersion('test-spec', testVersion.id);
169 |
170 |       expect(retrieved).toBeDefined();
171 |       expect(retrieved!.id).toBe(testVersion.id);
172 |       expect(retrieved!.specification).toEqual(testSpec);
173 |     });
174 |
175 |     it('should return null for non-existent version', async () => {
176 |       const retrieved = await versionControl.getVersion('test-spec', 'non-existent');
177 |
178 |       expect(retrieved).toBeNull();
179 |     });
180 |
181 |     it('should get version history', () => {
182 |       const history = versionControl.getVersionHistory('test-spec');
183 |
184 |       expect(history).toHaveLength(1);
185 |       expect(history[0].id).toBe(testVersion.id);
186 |     });
187 |
188 |     it('should get latest version', () => {
189 |       const latest = versionControl.getLatestVersion('test-spec');
190 |
191 |       expect(latest).toBeDefined();
192 |       expect(latest!.id).toBe(testVersion.id);
193 |     });
194 |
195 |     it('should get current version ID', () => {
196 |       const currentId = versionControl.getCurrentVersion('test-spec');
197 |
198 |       expect(currentId).toBe(testVersion.id);
199 |     });
200 |   });
201 |
202 |   describe('Version Comparison', () => {
203 |     let version1: VersionEntry;
204 |     let version2: VersionEntry;
```

```
205 |
206 |     beforeEach(async () => {
207 |       version1 = await versionControl.createVersion('test-spec', testSpec);
208  |               version2  =  await  versionControl.createVersion('test-spec',
testSpecModified);
209 |     });
210 |
211 |     it('should compare two versions', async () => {
212  |           const  diff  =  await  versionControl.compareVersions('test-spec',
version1.id, version2.id);
213 |
214 |       expect(diff).toBeDefined();
215 |       expect(diff.specId).toBe('test-spec');
216 |       expect(diff.fromVersion).toBe(version1.id);
217 |       expect(diff.toVersion).toBe(version2.id);
218 |        expect(diff.changes).toHaveLength(1); // One modification (the /users
path changed)
219 |     });
220 |
221 |     it('should detect modifications', async () => {
222  |           const  diff  =  await  versionControl.compareVersions('test-spec',
version1.id, version2.id);
223 |
224 |       const modification = diff.changes.find(c => c.type === 'modification');
225 |       expect(modification).toBeDefined();
226 |       expect(modification!.path).toBe('paths./users');
227 |     });
228 |
229 |     it('should detect additions', async () => {
230  |           const  diff  =  await  versionControl.compareVersions('test-spec',
version1.id, version2.id);
231 |
232 |       // The POST endpoint is an addition to the /users path
233 |       const changes = diff.changes.filter(c => c.type === 'modification');
234 |       expect(changes.length).toBeGreaterThan(0);
235 |     });
236 |   });
237 |
238 |   describe('Rollback', () => {
239 |     let version1: VersionEntry;
240 |     let version2: VersionEntry;
241 |
242 |     beforeEach(async () => {
243 |       version1 = await versionControl.createVersion('test-spec', testSpec);
244  |               version2  =  await  versionControl.createVersion('test-spec',
testSpecModified);
245 |     });
```

```
246 |
247 |     it('should rollback to previous version', async () => {
248 |       const rollbackVersion = await versionControl.rollback('test-spec', version1.id);
249 |
250 |       expect(rollbackVersion).toBeDefined();
251 |       expect(rollbackVersion.specification).toEqual(testSpec);
252 |       expect(rollbackVersion.metadata.message).toContain('Rollback to version');
253 |       expect(rollbackVersion.metadata.tags).toContain('rollback');
254 |     });
255 |
256 |     it('should emit rollback event', async () => {
257 |       const eventSpy = jest.fn();
258 |       versionControl.on('rollback', eventSpy);
259 |
260 |       const rollbackVersion = await versionControl.rollback('test-spec', version1.id);
261 |
262 |       expect(eventSpy).toHaveBeenCalledWith({
263 |         specId: 'test-spec',
264 |         targetVersion: version1,
265 |         rollbackVersion
266 |       });
267 |     });
268 |
269 |     it('should fail rollback to non-existent version', async () => {
270 |       await expect(
271 |         versionControl.rollback('test-spec', 'non-existent')
272 |       ).rejects.toThrow('Target version non-existent not found');
273 |     });
274 |   });
275 |
276 |   describe('Branching', () => {
277 |     let testVersion: VersionEntry;
278 |
279 |     beforeEach(async () => {
280 |       testVersion = await versionControl.createVersion('test-spec', testSpec);
281 |     });
282 |
283 |     it('should create a branch', async () => {
284 |       await versionControl.createBranch('test-spec', 'feature-branch', testVersion.id);
285 |
286 |       expect(mockLogger.info).toHaveBeenCalledWith('Created branch feature-branch for spec test-spec');
287 |     });
```

```
288 |
289 |     it('should emit branchCreated event', async () => {
290 |       const eventSpy = jest.fn();
291 |       versionControl.on('branchCreated', eventSpy);
292 |
293 |           await versionControl.createBranch('test-spec', 'feature-branch',
testVersion.id);
294 |
295 |       expect(eventSpy).toHaveBeenCalledWith({
296 |         specId: 'test-spec',
297 |         branchName: 'feature-branch',
298 |         sourceVersion: testVersion
299 |       });
300 |     });
301 |   });
302 |
303 |   describe('Conflict Detection', () => {
304 |     let version1: VersionEntry;
305 |     let version2: VersionEntry;
306 |
307 |     beforeEach(async () => {
308 |       version1 = await versionControl.createVersion('test-spec', testSpec);
309 |               version2 = await versionControl.createVersion('test-spec',
testSpecModified);
310 |     });
311 |
312 |     it('should detect conflicts between versions', async () => {
313 |         const conflicts = await versionControl.detectConflicts('test-spec',
version1.id, version2.id);
314 |
315 |       expect(conflicts).toBeDefined();
316 |       expect(Array.isArray(conflicts)).toBe(true);
317 |     });
318 |   });
319 |
320 |   describe('Statistics', () => {
321 |     beforeEach(async () => {
322 |       await versionControl.createVersion('spec1', testSpec);
323 |       await versionControl.createVersion('spec1', testSpecModified);
324 |       await versionControl.createVersion('spec2', testSpec);
325 |     });
326 |
327 |     it('should provide version control statistics', () => {
328 |       const stats = versionControl.getStatistics();
329 |
330 |       expect(stats.totalSpecs).toBe(2);
331 |       expect(stats.totalVersions).toBe(3);
```

```
332 |       expect(stats.averageVersionsPerSpec).toBe(1.5);
333 |       expect(stats.oldestVersion).toBeInstanceOf(Date);
334 |       expect(stats.newestVersion).toBeInstanceOf(Date);
335 |     });
336 |   });
337 |
338 |   describe('Error Handling', () => {
339 |     it('should throw error when not initialized', () => {
340 |       const uninitializedVC = new VersionControlManager(config, mockLogger as
any);
341 |
342 |       |                                                    expect(()       =>
uninitializedVC.getVersionHistory('test-spec')).toThrow('Version control system not
initialized');
343 |     });
344 |
345 |     it('should handle storage errors gracefully', async () => {
346 |       // Mock file system error
347 |       const originalWriteFile = fs.writeFile;
348 |       (fs.writeFile as any) = jest.fn().mockRejectedValue(new Error('Storage
error'));
349 |
350 |       await expect(
351 |         versionControl.createVersion('test-spec', testSpec)
352 |       ).rejects.toThrow();
353 |
354 |       // Restore original function
355 |       (fs.writeFile as any) = originalWriteFile;
356 |     });
357 |   });
358 | });
359 |
360 | describe('FileSystemProvider', () => {
361 |   let provider: FileSystemProvider;
362 |   let testDir: string;
363 |
364 |   beforeEach(async () => {
365 |     testDir = path.join(__dirname, '../../temp/fs-provider-test');
366 |     await fs.mkdir(testDir, { recursive: true });
367 |
368 |     provider = new FileSystemProvider({
369 |       basePath: testDir,
370 |       structure: 'hierarchical'
371 |     }, mockLogger as any);
372 |
373 |     await provider.initialize();
374 |   });
```

```
375 |
376 |   afterEach(async () => {
377 |     try {
378 |       await fs.rm(testDir, { recursive: true, force: true });
379 |     } catch (error) {
380 |       // Ignore cleanup errors
381 |     }
382 |   });
383 |
384 |   describe('File Operations', () => {
385 |     it('should write and read files', async () => {
386 |       const testPath = path.join(testDir, 'test.txt');
387 |       const testContent = 'Hello, World!';
388 |
389 |       await provider.writeFile(testPath, testContent);
390 |       const readContent = await provider.readFile(testPath);
391 |
392 |       expect(readContent).toBe(testContent);
393 |     });
394 |
395 |     it('should list files in directory', async () => {
396 |       await provider.writeFile(path.join(testDir, 'file1.txt'), 'content1');
397 |       await provider.writeFile(path.join(testDir, 'file2.txt'), 'content2');
398 |
399 |       const files = await provider.listFiles(testDir);
400 |
401 |       expect(files).toContain('file1.txt');
402 |       expect(files).toContain('file2.txt');
403 |     });
404 |
405 |     it('should delete files', async () => {
406 |       const testPath = path.join(testDir, 'test.txt');
407 |       await provider.writeFile(testPath, 'content');
408 |
409 |       await provider.deleteFile(testPath);
410 |
411 |       await expect(fs.access(testPath)).rejects.toThrow();
412 |     });
413 |   });
414 |
415 |   describe('Version Storage', () => {
416 |     it('should store and retrieve versions', async () => {
417 |       const version: VersionEntry = {
418 |         id: 'test-version-id',
419 |         specId: 'test-spec',
420 |         version: '1.0.0',
421 |         specification: testSpec,
```

```
422 |         hash: 'test-hash',
423 |         timestamp: new Date(),
424 |         metadata: {
425 |           author: 'test-user',
426 |           message: 'Test version',
427 |           tags: []
428 |         }
429 |       };
430 |
431 |       await provider.storeVersion(version);
432 |       const retrieved = await provider.getVersion('test-spec', 'test-version-id');
433 |
434 |       expect(retrieved).toBeDefined();
435 |       expect(retrieved!.id).toBe(version.id);
436 |       expect(retrieved!.specification).toEqual(testSpec);
437 |     });
438 |   });
439 | });
440 |
```

/tests/unit/windsurf-adapter.test.ts:

```
  1 | /**
  2 |  * Unit tests for Windsurf Tool Adapter
  3 |  */
  4 |
  5 | import { WindsurfAdapter, WindsurfAdapterConfig, WindsurfContextData } from '../../src/adapters/windsurf';
  6 | import { ValidationEngine } from '../../src/core/validation';
  7 | import { ContextEnhancer } from '../../src/core/context';
  8 | import { SpecParser } from '../../src/core/parser';
  9 |
 10 | // Mock the core modules
 11 | jest.mock('../../src/core/validation');
 12 | jest.mock('../../src/core/context');
 13 | jest.mock('../../src/core/parser');
 14 | jest.mock('../../src/utils/logger');
 15 |
 16 | describe('WindsurfAdapter', () => {
 17 |   let adapter: WindsurfAdapter;
 18 |   let mockConfig: WindsurfAdapterConfig;
 19 |   let mockValidationEngine: jest.Mocked<ValidationEngine>;
 20 |   let mockContextEnhancer: jest.Mocked<ContextEnhancer>;
 21 |   let mockSpecParser: jest.Mocked<SpecParser>;
```

```
22 |
23 |   beforeEach(() => {
24 |     // Reset all mocks
25 |     jest.clearAllMocks();
26 |     jest.clearAllTimers();
27 |     jest.useFakeTimers();
28 |
29 |     // Create mock config
30 |     mockConfig = {
31 |       enableRealTimeValidation: true,
32 |       contextInjectionMode: 'auto',
33 |       validationTriggers: ['save', 'change'],
34 |       apiEndpoint: 'https://api.windsurf.com',
35 |       serviceKey: 'test-service-key',
36 |       workspaceRoot: '/test/workspace',
37 |       mcpServerConfig: {
38 |         servers: {
39 |           github: {
40 |             command: 'npx',
41 |             args: ['-y', '@modelcontextprotocol/server-github'],
42 |             env: { GITHUB_PERSONAL_ACCESS_TOKEN: 'test-token' }
43 |           }
44 |         }
45 |       }
46 |     };
47 |
48 |     // Setup mocks for core modules
49 |     const MockedValidationEngine = ValidationEngine as jest.MockedClass<typeof
ValidationEngine>;
50 |     const MockedContextEnhancer = ContextEnhancer as jest.MockedClass<typeof
ContextEnhancer>;
51 |     const MockedSpecParser = SpecParser as jest.MockedClass<typeof SpecParser>;
52 |
53   |             mockValidationEngine = new MockedValidationEngine() as
jest.Mocked<ValidationEngine>;
54   |              mockContextEnhancer = new MockedContextEnhancer() as
jest.Mocked<ContextEnhancer>;
55 |     mockSpecParser = new MockedSpecParser() as jest.Mocked<SpecParser>;
56 |
57 |     // Mock the initialize methods
58 |     mockValidationEngine.initialize = jest.fn().mockResolvedValue(undefined);
59 |     mockContextEnhancer.initialize = jest.fn().mockResolvedValue(undefined);
60 |
61 |     // Mock the parseSpec method
62 |     mockSpecParser.parseSpec = jest.fn();
63 |
64 |     // Mock the validateInterface method
```

```
 65 |     mockValidationEngine.validateInterface = jest.fn();
 66 |
 67 |     // Create adapter instance with mocked dependencies
 68 |     adapter = new WindsurfAdapter(mockConfig, mockValidationEngine,
mockContextEnhancer, mockSpecParser);
 69 |   });
 70 |
 71 |   afterEach(() => {
 72 |     jest.useRealTimers();
 73 |     adapter.dispose();
 74 |   });
 75 |
 76 |   describe('Initialization', () => {
 77 |     it('should initialize successfully with valid config', async () => {
 78 |       const initSpy = jest.fn();
 79 |       adapter.on('initialized', initSpy);
 80 |
 81 |       await adapter.initialize();
 82 |
 83 |       expect(mockValidationEngine.initialize).toHaveBeenCalled();
 84 |       expect(mockContextEnhancer.initialize).toHaveBeenCalled();
 85 |       expect(initSpy).toHaveBeenCalled();
 86 |     });
 87 |
 88 |     it('should setup MCP servers when configured', async () => {
 89 |       await adapter.initialize();
 90 |
 91 |       const stats = adapter.getStatistics();
 92 |       expect(stats.mcpServersCount).toBe(1);
 93 |       expect(stats.mcpServers).toContain('github');
 94 |     });
 95 |
 96 |     it('should setup real-time monitoring when enabled', async () => {
 97 |       const monitoringSpy = jest.fn();
 98 |       adapter.on('monitoring', monitoringSpy);
 99 |
100 |       await adapter.initialize();
101 |
102 |       // Fast-forward time to trigger monitoring
103 |       jest.advanceTimersByTime(6000);
104 |
105 |       expect(monitoringSpy).toHaveBeenCalled();
106 |     });
107 |
108 |     it('should handle initialization errors gracefully', async () => {
109 |       mockValidationEngine.initialize.mockRejectedValue(new Error('Init
failed'));
```

```
110 |
111 |       await expect(adapter.initialize()).rejects.toThrow('Init failed');
112 |     });
113 |
114 |     it('should not setup MCP servers when not configured', async () => {
115 |       const configWithoutMCP = { ...mockConfig };
116 |       delete configWithoutMCP.mcpServerConfig;
117 |
118 |             const mockValidationEngineLocal = new ValidationEngine() as
jest.Mocked<ValidationEngine>;
119 |              const mockContextEnhancerLocal = new ContextEnhancer() as
jest.Mocked<ContextEnhancer>;
120 |       const mockSpecParserLocal = new SpecParser() as jest.Mocked<SpecParser>;
121 |
122 |                                   mockValidationEngineLocal.initialize    =
jest.fn().mockResolvedValue(undefined);
123 |                                    mockContextEnhancerLocal.initialize    =
jest.fn().mockResolvedValue(undefined);
124 |
125 |           const adapterWithoutMCP = new WindsurfAdapter(configWithoutMCP,
mockValidationEngineLocal, mockContextEnhancerLocal, mockSpecParserLocal);
126 |       await adapterWithoutMCP.initialize();
127 |
128 |       const stats = adapterWithoutMCP.getStatistics();
129 |       expect(stats.mcpServersCount).toBe(0);
130 |
131 |       adapterWithoutMCP.dispose();
132 |     });
133 |   });
134 |
135 |   describe('Document Validation', () => {
136 |     const mockContextData: WindsurfContextData = {
137 |       documentPath: '/test/api.yaml',
138 |       content: `
139 | openapi: 3.0.0
140 | info:
141 |   title: Test API
142 |   version: 1.0.0
143 | paths:
144 |   /users:
145 |     get:
146 |       responses:
147 |         '200':
148 |           description: Success
149 |       `,
150 |       language: 'yaml',
151 |       projectInfo: {
```

```
152 |         name: 'test-project',
153 |         version: '1.0.0',
154 |         type: 'api'
155 |       },
156 |       ideInfo: {
157 |         type: 'vscode',
158 |         version: '1.89.0'
159 |       }
160 |     };
161 |
162 |     beforeEach(async () => {
163 |       await adapter.initialize();
164 |     });
165 |
166 |     it('should validate API specification documents successfully', async () =>
{
167 |       // Mock successful parsing
168 |       mockSpecParser.parseSpec.mockResolvedValue({
169 |         success: true,
170 |         spec: { openapi: '3.0.0', info: { title: 'Test API' } },
171 |         errors: []
172 |       });
173 |
174 |       // Mock successful validation
175 |       mockValidationEngine.validateInterface.mockResolvedValue({
176 |         isValid: true,
177 |         errors: [],
178 |         warnings: [],
179 |         suggestions: []
180 |       });
181 |
182 |       const result = await adapter.validateDocument(mockContextData);
183 |
184 |       expect(result.isValid).toBe(true);
185 |       expect(result.errors).toHaveLength(0);
186 |       expect(result.metadata?.model).toBe('windsurf-validator');
187 |       expect(result.metadata?.tokensUsed).toBeGreaterThan(0);
188 |     });
189 |
190 |     it('should handle validation errors properly', async () => {
191 |       // Mock parsing failure
192 |       mockSpecParser.parseSpec.mockResolvedValue({
193 |         success: false,
194 |         spec: null,
195 |         errors: ['Invalid YAML syntax']
196 |       });
197 |
```

```
198 |       const result = await adapter.validateDocument(mockContextData);
199 |
200 |       expect(result.isValid).toBe(false);
201 |       expect(result.errors).toContain('Failed to parse API specification');
202 |     });
203 |
204 |     it('should cache validation results', async () => {
205 |       mockSpecParser.parseSpec.mockResolvedValue({
206 |         success: true,
207 |         spec: { openapi: '3.0.0' },
208 |         errors: []
209 |       });
210 |
211 |       mockValidationEngine.validateInterface.mockResolvedValue({
212 |         isValid: true,
213 |         errors: [],
214 |         warnings: [],
215 |         suggestions: []
216 |       });
217 |
218 |       // First validation
219 |       const result1 = await adapter.validateDocument(mockContextData);
220 |
221 |       // Second validation with same content
222 |       const result2 = await adapter.validateDocument(mockContextData);
223 |
224 |       expect(mockSpecParser.parseSpec).toHaveBeenCalledTimes(1);
225 |       expect(result1.timestamp).toBe(result2.timestamp);
226 |     });
227 |
228 |     it('should emit validation events', async () => {
229 |       const validationSpy = jest.fn();
230 |       adapter.on('validationResult', validationSpy);
231 |
232 |       mockSpecParser.parseSpec.mockResolvedValue({
233 |         success: true,
234 |         spec: { openapi: '3.0.0' },
235 |         errors: []
236 |       });
237 |
238 |       mockValidationEngine.validateInterface.mockResolvedValue({
239 |         isValid: true,
240 |         errors: [],
241 |         warnings: [],
242 |         suggestions: []
243 |       });
244 |
```

```
245 |       await adapter.validateDocument(mockContextData);
246 |
247 |       expect(validationSpy).toHaveBeenCalledWith(
248 |         expect.objectContaining({
249 |           isValid: true,
250 |           errors: [],
251 |           warnings: []
252 |         })
253 |       );
254 |     });
255 |
256 |     it('should handle non-API documents', async () => {
257 |       const nonApiContext = {
258 |         ...mockContextData,
259 |         content: 'console.log("Hello World");',
260 |         language: 'javascript'
261 |       };
262 |
263 |       const result = await adapter.validateDocument(nonApiContext);
264 |
265 |       expect(result.isValid).toBe(true);
266 |       expect(mockSpecParser.parseSpec).not.toHaveBeenCalled();
267 |     });
268 |
269 |     it('should handle validation exceptions', async () => {
270 |       mockSpecParser.parseSpec.mockRejectedValue(new Error('Parser error'));
271 |
272 |       const result = await adapter.validateDocument(mockContextData);
273 |
274 |       expect(result.isValid).toBe(false);
275 |       expect(result.errors[0]).toContain('Validation error: Parser error');
276 |     });
277 |   });
278 |
279 |   describe('Context Injection', () => {
280 |     const mockContextData: WindsurfContextData = {
281 |       documentPath: '/test/component.tsx',
282 |       content: 'export const Component = () => <div>Hello</div>;',
283 |       language: 'typescript',
284 |       ideInfo: {
285 |         type: 'vscode',
286 |         version: '1.89.0'
287 |       }
288 |     };
289 |
290 |     beforeEach(async () => {
291 |       await adapter.initialize();
```

```
292 |     });
293 |
294 |     it('should inject context successfully', async () => {
295 |       const contextSpy = jest.fn();
296 |       adapter.on('contextInjected', contextSpy);
297 |
298 |       await adapter.injectContext(mockContextData);
299 |
300 |       expect(contextSpy).toHaveBeenCalledWith(
301 |         expect.objectContaining({
302 |           contextData: mockContextData,
303 |           context: expect.objectContaining({
304 |             id: expect.stringMatching(/^windsurf-context-\d+$/),
305 |             type: 'api-validation',
306 |             language: 'typescript',
307 |             ideType: 'vscode'
308 |           })
309 |         })
310 |       );
311 |     });
312 |
313 |     it('should handle context injection errors', async () => {
314 |       // Force an error by making the adapter inactive
315 |       adapter.dispose();
316 |
317 |       await expect(adapter.injectContext(mockContextData)).rejects.toThrow();
318 |     });
319 |   });
320 |
321 |   describe('Analytics Queries', () => {
322 |     beforeEach(async () => {
323 |       await adapter.initialize();
324 |     });
325 |
326 |     it('should query analytics successfully', async () => {
327 |       const analyticsSpy = jest.fn();
328 |       adapter.on('analyticsQueried', analyticsSpy);
329 |
330 |       const result = await adapter.queryAnalytics(
331 |         '2024-01-01T00:00:00Z',
332 |         '2024-01-31T23:59:59Z',
333 |         { groupName: 'test-group' }
334 |       );
335 |
336 |       expect(result).toHaveProperty('cascade_lines');
337 |       expect(result).toHaveProperty('cascade_runs');
338 |       expect(result).toHaveProperty('tool_usage');
```

```
339 |       expect(analyticsSpy).toHaveBeenCalledWith(result);
340 |     });
341 |
342 |     it('should require service key and API endpoint for analytics', async () =>
{
343 |       const configWithoutAuth = { ...mockConfig };
344 |       delete configWithoutAuth.serviceKey;
345 |       delete configWithoutAuth.apiEndpoint;
346 |
347 |             const mockValidationEngineLocal = new ValidationEngine() as
jest.Mocked<ValidationEngine>;
348 |              const mockContextEnhancerLocal = new ContextEnhancer() as
jest.Mocked<ContextEnhancer>;
349 |       const mockSpecParserLocal = new SpecParser() as jest.Mocked<SpecParser>;
350 |
351 |                                   mockValidationEngineLocal.initialize    =
jest.fn().mockResolvedValue(undefined);
352 |                                    mockContextEnhancerLocal.initialize    =
jest.fn().mockResolvedValue(undefined);
353 |
354 |          const adapterWithoutAuth = new WindsurfAdapter(configWithoutAuth,
mockValidationEngineLocal, mockContextEnhancerLocal, mockSpecParserLocal);
355 |       await adapterWithoutAuth.initialize();
356 |
357 |       await expect(
358 |                   adapterWithoutAuth.queryAnalytics('2024-01-01T00:00:00Z',
'2024-01-31T23:59:59Z')
359 |       ).rejects.toThrow('Service key and API endpoint required');
360 |
361 |       adapterWithoutAuth.dispose();
362 |     });
363 |   });
364 |
365 |   describe('Cache Management', () => {
366 |     beforeEach(async () => {
367 |       await adapter.initialize();
368 |     });
369 |
370 |     it('should clear validation cache', async () => {
371 |       // Add something to cache first
372 |       const mockContextData: WindsurfContextData = {
373 |         documentPath: '/test/api.yaml',
374 |         content: `
375 | openapi: 3.0.0
376 | info:
377 |   title: Test API
378 |   version: 1.0.0
```

```
paths:
  /users:
    get:
      responses:
        '200':
          description: Success
        `,
        language: 'yaml'
      };

      mockSpecParser.parseSpec.mockResolvedValue({
        success: true,
        spec: { openapi: '3.0.0' },
        errors: []
      });

      mockValidationEngine.validateInterface.mockResolvedValue({
        isValid: true,
        errors: [],
        warnings: [],
        suggestions: []
      });

      await adapter.validateDocument(mockContextData);

      let stats = adapter.getStatistics();
      expect(stats.cacheSize).toBeGreaterThan(0);

      adapter.clearValidationCache();

      stats = adapter.getStatistics();
      expect(stats.cacheSize).toBe(0);
    });
  });

  describe('Statistics and Monitoring', () => {
    it('should provide accurate statistics', async () => {
      await adapter.initialize();

      const stats = adapter.getStatistics();

      expect(stats).toHaveProperty('isActive', true);
      expect(stats).toHaveProperty('cacheSize', 0);
      expect(stats).toHaveProperty('mcpServersCount', 1);
      expect(stats).toHaveProperty('config');
      expect(stats).toHaveProperty('mcpServers');
      expect(stats.mcpServers).toContain('github');
```

```
426 |     });
427 |
428 |     it('should show inactive status after disposal', () => {
429 |       adapter.dispose();
430 |
431 |       const stats = adapter.getStatistics();
432 |       expect(stats.isActive).toBe(false);
433 |       expect(stats.cacheSize).toBe(0);
434 |       expect(stats.mcpServersCount).toBe(0);
435 |     });
436 |   });
437 |
438 |   describe('Configuration Handling', () => {
439 |     it('should handle minimal configuration', async () => {
440 |       const minimalConfig: WindsurfAdapterConfig = {
441 |         enableRealTimeValidation: false,
442 |         contextInjectionMode: 'disabled',
443 |         validationTriggers: []
444 |       };
445 |
446 |       const mockValidationEngineLocal = new ValidationEngine() as
jest.Mocked<ValidationEngine>;
447 |       const mockContextEnhancerLocal = new ContextEnhancer() as
jest.Mocked<ContextEnhancer>;
448 |       const mockSpecParserLocal = new SpecParser() as jest.Mocked<SpecParser>;
449 |
450 |       mockValidationEngineLocal.initialize =
jest.fn().mockResolvedValue(undefined);
451 |       mockContextEnhancerLocal.initialize =
jest.fn().mockResolvedValue(undefined);
452 |
453 |       const minimalAdapter = new WindsurfAdapter(minimalConfig,
mockValidationEngineLocal, mockContextEnhancerLocal, mockSpecParserLocal);
454 |       await minimalAdapter.initialize();
455 |
456 |       const stats = minimalAdapter.getStatistics();
457 |       expect(stats.config.enableRealTimeValidation).toBe(false);
458 |       expect(stats.config.contextInjectionMode).toBe('disabled');
459 |       expect(stats.mcpServersCount).toBe(0);
460 |
461 |       minimalAdapter.dispose();
462 |     });
463 |
464 |     it('should handle complex MCP server configurations', async () => {
465 |       const complexConfig = {
466 |         ...mockConfig,
467 |         mcpServerConfig: {
```

```
468 |           servers: {
469 |             github: {
470 |               command: 'npx',
471 |               args: ['-y', '@modelcontextprotocol/server-github'],
472 |               env: { GITHUB_PERSONAL_ACCESS_TOKEN: 'token1' }
473 |             },
474 |             filesystem: {
475 |               command: 'npx',
476 |                     args: ['-y', '@modelcontextprotocol/server-filesystem',
'/allowed/path'],
477 |               env: { NODE_ENV: 'development' }
478 |             }
479 |           }
480 |         }
481 |       };
482 |
483 |             const mockValidationEngineLocal = new ValidationEngine() as
jest.Mocked<ValidationEngine>;
484 |             const mockContextEnhancerLocal = new ContextEnhancer() as
jest.Mocked<ContextEnhancer>;
485 |       const mockSpecParserLocal = new SpecParser() as jest.Mocked<SpecParser>;
486 |
487 |                                   mockValidationEngineLocal.initialize =
jest.fn().mockResolvedValue(undefined);
488 |                                    mockContextEnhancerLocal.initialize =
jest.fn().mockResolvedValue(undefined);
489 |
490 |             const complexAdapter = new WindsurfAdapter(complexConfig,
mockValidationEngineLocal, mockContextEnhancerLocal, mockSpecParserLocal);
491 |       await complexAdapter.initialize();
492 |
493 |       const stats = complexAdapter.getStatistics();
494 |       expect(stats.mcpServersCount).toBe(2);
495 |       expect(stats.mcpServers).toContain('github');
496 |       expect(stats.mcpServers).toContain('filesystem');
497 |
498 |       complexAdapter.dispose();
499 |     });
500 |   });
501 | });
502 |
```

/tsconfig.json:

```
1 | {
```

```
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2022"],
    "outDir": "./build",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "removeComments": false,
    "noImplicitAny": true,
    "noImplicitReturns": true,
    "noImplicitThis": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noUncheckedIndexedAccess": true,
    "allowUnusedLabels": false,
    "allowUnreachableCode": false,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "verbatimModuleSyntax": true
  },
  "include": [
    "src/**/*"
  ],
  "exclude": [
    "node_modules",
    "build",
    "**/*.test.ts",
    "**/*.spec.ts"
  ],
  "ts-node": {
    "esm": true
  }
}

```
